

AI-Based Retinal Disease Detection Using Deep Learning with Heatmap Visualisation

A project report submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING(AI&ML)

Submitted by

BOINA NAMITHA (5221412006)

RONGALI PRAVEEN KUMAR (5221412043)

YADLA AJAY (5221412055)

KOYYANA DURGA PRASAD (5221412025)

Under the Esteemed Guidance of

Mrs. K.Kausalya B. Tech, M. Tech

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(AI&ML)

ENGINEERING AND TECHNOLOGY PROGRAM

GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE AND PG COURSES (A)

Rushikonda, Visakhapatnam - 45

(Approved by AICTE| Accredited by NBA| Accredited by NAAC| Affiliated to Andhra University)

An ISO 9001:2015 Certified Institution

2022-2026

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING(AI&ML)

**GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE AND PG
COURSES(A)**

Rushikonda, Visakhapatnam - 530045



CERTIFICATE

This is to certify that the project report entitled “AI-Based Retinal Disease Detection Using Deep Learning with Heatmap Visualisation” being submitted by BOINA NAMITHA (5221412006), RONGALI PRAVEEN KUMAR (5221412043), YADLA AJAY (5221412055), KOYYANA DURGA PRASAD (5221412025) in the partial fulfilment for the award of the Degree of Bachelor of Technology in Computer Science and Engineering (AI&ML) to the Gayatri Vidya Parishad College for Degree and PG Courses (A), Visakhapatnam is a record of Bonafide work carried out under my guidance and supervision.

Project Guide

K. Kausalya
Assistant Professor
B. Tech, M. Tech

Head of the Department

Dr. N.V. Ramana Murty
Professor
M. Tech, Ph. D

External Examiner

VISION AND MISSION OF THE INSTITUTE

VISION

Creating Human Excellence for a Better Society

MISSION

Unfold into a World class organization with strong academic and research base, producing responsible citizens to cater to the changing needs of the society.

VISION AND MISSION OF THE DEPARTMENT

VISION

“To enhance employability, foster innovative research, and nurture entrepreneurial thinking by implementing effective teaching and learning practices, aimed at developing cutting-edge technology solutions that benefit both industry and society.”

MISSION

“Delivering high-quality, value-driven technical education to shape technology professionals who think creatively and lead with inspiration.

Utilizing rational thinking to design and develop cutting-edge products in collaboration with industry stakeholders, aligning with global standards and demands.

Enhancing core skills in the fields of Artificial Intelligence and Machine Learning”.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

- **PEO1: Lead in diverse professional roles at reputed industries and address challenges by applying principles of computer science, mathematics, artificial intelligence, machine learning, and scientific investigation to deliver industry-ready solutions using advanced technologies.**
- **PEO2: Exhibit effective communication, teamwork, leadership skills, and ethical behaviour while achieving professional growth through lifelong learning and continuing education.**
- **PEO3: Pursue advanced education in leading graduate programs across engineering and interdisciplinary fields to excel as researchers, experts, educators, and entrepreneurs, while recognizing the importance of and actively engaging in lifelong learning and professional development.**

PROGRAM SPECIFIC OUTCOMES (PSOs)

- **PSO 1: Design and develop optimized solutions for computationally intensive applications.**
- **PSO 2: Apply tools and techniques of Artificial Intelligence and Machine Learning to address multidisciplinary challenges**

DECLARATION

We hereby declare that the project entitled “**AI-Based Retinal Disease Detection Using Deep Learning with Heatmap Visualisation**” submitted in partial fulfilment of the requirements for the award of Bachelor of Technology in Computer Science and Engineering (AI&ML), to Engineering and Technology Program, Gayatri Vidya Parishad College for Degree & PG Courses (A). We assure that this project is not submitted in any other University or College.

Name & Signature of the Students

B. NAMITHA (5221412006)

R. PRAVEEN KUMAR (5221412043)

Y. AJAY (5221412055)

K. DURGA PRASAD (5221412025)

ACKNOWLEDGEMENTS

With great pleasure, we want to take this opportunity to express our heartfelt gratitude to all the people who helped in making main project work a grand success.

Firstly, gratitude to **K. Kausalya**, Assistant Professor, for his constant guidance throughout our main project work.

We are grateful to **Dr. D.S.S.N. Raju**, Assistant Professor, Department of Computer Science and Engineering (AI&ML) as project coordinator, for his valuable suggestions and guidance given by him during the execution of this main project.

We would like to thank **Dr. N.V. Ramana Murty**, Professor, Head of the Department of Computer Science and Engineering (AI&ML), for being moral support throughout the period of our study.

We are highly indebted to **Dr. P.V. Vinay**, Director – E&T and **Prof. K.S. Bose**, Principal for giving us the permission to carry out this main project in the college.

We would like to thank all the Teaching and Non-Teaching staff of CSE and CSE (AI&ML) Department for sharing their knowledge with us.

BOINA NAMITHA	(5221412027)
RONGALI PRAVEEN KUMAR	(5221412015)
YADLA AJAY	(5221412016)
KOYYANA DURGA PRASAD	(5221412003)

ABSTRACT

Eye diseases such as Cataract, Glaucoma, and Diabetic Retinopathy are major causes of vision loss worldwide. Early detection and timely treatment are essential to prevent permanent blindness. This project presents an AI based Eye Retinal Disease Classification using Deep learning and heatmap for visualization that uses retinal image analysis to identify different eye conditions accurately and efficiently.

The system allows users to upload a retinal image through a web interface. The uploaded image undergoes preprocessing steps such as resizing and normalization to ensure consistency and improve accuracy. A Convolutional Neural Network (CNN) is used to extract important visual features from the retinal image and classify it into predefined categories such as Cataract, Glaucoma, Diabetic Retinopathy, or Normal.

The system also calculates a confidence score to indicate the reliability of the prediction. To improve transparency and trust, a visual heatmap is generated to highlight the affected regions that influenced the model's decision. The results, including the predicted disease, confidence score, and visual explanation, are compiled into a structured medical report that can be viewed or downloaded.

In addition to disease prediction, the system provides general eye care routine suggestions, nearby hospital recommendations, and an appointment booking option to assist users in seeking further medical consultation.

Overall, this project demonstrates how artificial intelligence can support early detection of eye diseases, improve awareness, and assist in timely medical intervention, thereby contributing to better eye healthcare management.

TABLE OF CONTENTS

CHAPTER	Page No
1.Introduction	1
1.1 Introduction to ML	3
1.2 Applications	
1.3 Introduction to AI-Based Eye Retinal Disease Detection System	4
2. Literature Survey	6
3.Requirements and Analysis	9
3.1 Existing System	10
3.2 Proposed System	10
3.3 Functional Requirements	12
3.4 Non-Functional Requirements	13
3.5 Analysis	21
4. System Design	
4.1 Design Goals	24
4.2 System Architecture	26
4.3 System Design	26
4.4 UML Diagrams	27
4.5 Data Flow Diagrams	31
4.6 Flow Chart	33
5.Implementation	
5.1 Coding Approach	36
5.2 Information Handling	36
5.3 Programming Style	37
5.4 Verification and Validation	37
5.5 Implementation of Project	38
5.6 System Workflow and Process Flow	39
6. Testing	
6.1 Testing Activities	41
6.2 Testing Types	41
6.3 Test Cases	44

7.Results	
7.1 Home Page	47
7.2 Login Page	48
7.3 Eye Disease Detection	48
7.4 Disease Detection Page	43
7.5 Routine Page	50
7.6 Nearest hospitals	51
7.7 Book Appointment	52
8. Sample Code	
8.1 Code for Model Training	54
8.2 Code for detection	56
8.3 Code for landing page	65
9. Conclusion	75
10. References	78

LIST OF TABLES

Title	Page No
6.3 Test cases	44

LIST OF FIGURES

Title	Page No
3.4.1.1 Computer Processor Chip	14
3.4.1.2 SQLite Logo	15
3.4.1.3 User Devices	16
3.4.2.1 Windows / Mac os	17
3.4.2.2 Python Language Logo	18
3.4.2.3 Tensorflow & Keras Logo	19
3.4.2.4 CNN Architecture	20
3.4.2.5 Layered approach of HTML, CSS, JS	21
3.4.2.6 OpenStreetMap Logo	21
4.3.1 System's Block Diagram	26
4.4.1.1 Use Case Diagram	28
4.4.2.1 Class Diagram	29
4.4.3.1 Activity Diagram	30
4.4.4.1 Sequence Diagram	31
4.5.1 DFD Level 0	32
4.5.2 DFD Level 1	33
4.6.1 Flow chart of the system	33
6.2.1 Black Box Testing	43
7.1.1 Home Page	42
7.2.1 Login Page	47
7.2.2 Signup Page	48
7.3.1 Eye Disease Detection	48
7.4.1 Prediction Result Page	49
7.4.2 Heatmap Result Page	49
7.5.1 Routine Page	50
7.5.2 Routine Result	50
7.6.1 Nearest Hospital	51
7.6.2 Nearest Hospital Result	51
7.7.1 Appointment Page	
7.7.2 Appointment Page e Result	52

INTRODUCTION

1.INTRODUCTION

1.1 Introduction to Machine Learning

Machine Learning is a key branch of Artificial Intelligence that enables computers to learn from data and make decisions without being explicitly programmed. Unlike traditional programming methods, where specific rules are written by developers, machine learning systems automatically identify patterns and relationships within data. This ability allows machines to improve their performance over time through experience, making them highly efficient for solving complex and dynamic problems.

The importance of machine learning has increased significantly due to the rapid growth of digital data and advancements in computational power. It is widely used across various domains such as healthcare, finance, education, and e-commerce. Applications of machine learning include image recognition, speech processing, fraud detection, recommendation systems, and predictive analytics. In the healthcare sector, machine learning is particularly valuable as it helps in analyzing medical data and supporting early diagnosis of diseases.

Machine learning techniques are generally categorized into three main types: supervised learning, unsupervised learning, and reinforcement learning. In supervised learning, the model is trained using labelled data where the correct output is known, making it suitable for tasks like classification and prediction. Unsupervised learning works with unlabelled data and focuses on identifying hidden patterns or groupings. Reinforcement learning, on the other hand, involves training a model through interaction with an environment, where it learns based on rewards and penalties.

A major advancement in this field is Deep Learning, which uses multi-layered neural networks to process complex data. Deep learning models, especially Convolutional Neural Networks (CNNs), are highly effective in image-related tasks as they automatically extract important features from images. This makes them highly suitable for applications such as medical image analysis, including the detection of eye diseases from retinal images.

1.2 Applications

The wide range of applications of Machine Learning extends across multiple domains, improving efficiency, accuracy, and decision-making. These applications are used in various sectors such as healthcare, finance, education, transportation, e-commerce, and natural language processing.

1.2.1 Healthcare Applications

Machine learning plays a crucial role in the healthcare sector by enabling early disease detection and accurate diagnosis. It is widely used in medical image analysis, where algorithms analyze X-rays, MRIs, and retinal images to detect diseases. It also assists in drug discovery, patient monitoring, and personalized treatment planning. In this project, machine learning is used to classify eye diseases such as cataract, glaucoma, and diabetic retinopathy from retinal images.

1.2.2 Finance Applications

In the finance sector, machine learning is used for fraud detection, risk assessment, and algorithmic trading. Banks and financial institutions use machine learning models to analyze transaction patterns and detect unusual activities. It is also used for credit scoring, customer segmentation, and predicting stock market trends, helping organizations make better financial decisions.

1.2.3 Education Applications

Machine learning is transforming the education sector by enabling personalized learning experiences. It helps in analyzing student performance and providing customized study recommendations. Online learning platforms use machine learning to suggest courses, automate grading, and detect plagiarism. It also supports intelligent tutoring systems that adapt to individual learning speeds.

1.2.3 Transportation Applications

Machine learning is widely used in transportation systems to improve safety and efficiency. It plays a key role in autonomous vehicles, traffic prediction, and route optimization. Ride-sharing services use machine learning algorithms to match drivers with passengers and estimate travel time. It also helps in monitoring traffic conditions and reducing congestion in smart cities.

1.2.4 E-Commerce and Marketing Applications

In e-commerce and marketing, machine learning is used to enhance customer experience and increase sales. Recommendation systems suggest products based on user behavior and preferences. It is also used for targeted advertising, customer segmentation, demand forecasting, and sentiment analysis. These applications help businesses understand customer needs and improve marketing strategies.

1.2.5 Natural Language Processing (NLP) Applications

Natural Language Processing is a branch of machine learning that focuses on the interaction between computers and human language. NLP applications include chatbots, language translation, sentiment analysis, and voice assistants. It enables machines to understand, interpret, and respond to human language, making communication between humans and computers more effective.

1.3 Introduction to AI-Based Retinal Disease Detection Using Deep Learning with Heatmap Visualisation

The rapid advancement of Artificial Intelligence has significantly transformed the healthcare sector, particularly in the field of medical imaging and disease diagnosis. Among various applications, retinal disease detection has gained considerable attention due to the increasing prevalence of vision-related disorders such as cataract, glaucoma, and diabetic retinopathy. Early detection of these diseases is crucial to prevent permanent vision loss; however, traditional diagnostic methods require skilled ophthalmologists and specialized equipment, which may not be easily accessible in remote or underdeveloped regions.

With the emergence of Deep Learning, automated retinal disease detection has become more efficient and accurate. Deep learning models, especially Convolutional Neural Networks (CNNs), can learn complex patterns from retinal images without the need for manual feature extraction. These models analyze input images, identify relevant features, and classify them into different disease categories. This approach reduces human effort and improves diagnostic consistency, making it highly suitable for large-scale screening systems.

An important aspect of AI-based medical systems is interpretability, which ensures that the model's decisions can be understood by medical professionals. This is achieved through heatmap visualization techniques, which highlight the regions of the retinal image that contribute most to the model's prediction. Heatmaps provide visual explanations by indicating areas of high importance, helping doctors verify the results and increasing trust in the system. Techniques such as Gradient-weighted Class Activation Mapping (Grad-CAM) are commonly used for this purpose.

The integration of deep learning with heatmap visualization not only improves diagnostic accuracy but also enhances transparency in decision-making. It allows clinicians to validate predictions and understand how the model is analyzing retinal images. This combination is particularly useful in critical applications like eye disease detection, where reliability and explainability are essential.

LITERATURE SURVEY

2.LITERATURE SURVEY

Recent advancements in Artificial Intelligence and Deep Learning have significantly contributed to the development of automated systems for retinal disease detection. Various research studies have explored different models and techniques to improve the accuracy and efficiency of eye disease classification using retinal images. This section reviews the key research works that form the foundation of the proposed system.

The first base paper, **“AI-Driven Eye Disease Classification Model (2023)”**, presents a highly accurate approach for detecting eye diseases using advanced hybrid techniques. The study utilizes denoising autoencoders for effective image preprocessing and feature extraction, followed by Single Shot Detector (SSD) for identifying important regions within retinal images. For classification, the model integrates Whale Optimization Algorithm with ShuffleNet V2, resulting in improved performance and reduced computational complexity. The proposed system achieved an impressive accuracy of 99% on benchmark datasets such as ODIR (Ocular Disease Intelligent Recognition) and EDC. Despite its high accuracy, the model involves complex optimization techniques, which may increase implementation difficulty and computational cost.[1]

The second base paper, **“Review of Eye Disease Detection Using Deep Learning (2024)”**, provides a comprehensive survey of various deep learning approaches used in retinal disease detection. The paper discusses widely used architectures such as CNN, ResNet, VGG, and EfficientNet, highlighting their strengths in image classification tasks. It also emphasizes the importance of preprocessing techniques like image resizing, normalization, and data augmentation to improve model performance. Additionally, the study identifies key challenges in the field, including limited dataset availability, class imbalance, and lack of interpretability in model predictions.[2]

From the analysis of these studies, it is evident that while deep learning models achieve high accuracy, there are still gaps that need to be addressed. Many existing systems focus primarily on performance metrics without providing clear

explanations for their predictions. This lack of transparency can reduce trust in real-world medical applications. Furthermore, the dependence on large datasets and complex architectures can limit the practical implementation of such systems in resource-constrained environments.

Based on the insights from the reviewed literature, the proposed system aims to develop an efficient and interpretable model for retinal disease classification. By incorporating deep learning techniques along with heatmap visualization, the system not only ensures accurate predictions but also provides visual explanations for better understanding and validation. This approach addresses the limitations identified in previous works and contributes to the development of reliable AI-based healthcare solutions.

REQUIREMENT ANALYSIS

3.REQUIREMENTS AND ANALYSIS

3.1 Existing system

- Traditional retinal disease detection mainly depends on manual examination by ophthalmologists using specialized equipment. This process is time- consuming and requires high expertise, making it less accessible in rural or resource-limited areas.
- With advancements in Deep Learning, automated systems using models like CNN, ResNet, and VGG have been developed for retinal image classification. These systems improve accuracy but often rely on complex architectures.
- Many existing models require large volumes of labeled data and high computational power for training and deployment. This makes them difficult to implement in real-time or low-resource environments.
- Most systems lack interpretability, functioning as “black box” models that do not explain how predictions are made. This reduces trust among medical professionals in critical healthcare applications.
- The performance of these systems is highly dependent on image quality, preprocessing techniques, and dataset balance. Poor-quality images or imbalanced datasets can lead to inaccurate or biased predictions.

3.2 Proposed System

- The proposed system utilizes advanced Deep Learning techniques to automatically detect and classify retinal diseases from input images. It reduces the need for manual diagnosis and provides faster and more accurate results.
- The system begins with retinal image acquisition, followed by preprocessing steps such as resizing, normalization, and noise reduction. These steps enhance image quality and ensure consistency for better model performance.
- A Convolutional Neural Network (CNN)-based model is used for feature extraction and classification of diseases such as cataract, glaucoma, and diabetic retinopathy.
- The model learns important patterns from retinal images to improve prediction

accuracy. To improve interpretability, the system integrates heatmap visualization techniques (such as Grad-CAM), which highlight the regions of the image that influence the model's decision. This helps doctors understand and validate the results.

- The proposed system is designed to be efficient and scalable, requiring less computational power compared to complex models. It can be deployed in real-world applications, including rural healthcare settings, for early disease detection.

3.2.1 Project scope

The scope of this project focuses on developing an intelligent system using Deep Learning for the early detection and classification of retinal diseases from medical images. It aims to provide accurate diagnosis along with heatmap visualization for better interpretability of results. The system also extends support by offering general recommendations, hospital information, and appointment booking features. This project can be applied in real-world healthcare systems, especially in rural and remote areas to improve accessibility and early diagnosis.

3.2.2 Project objectives

The system has some objectives to achieve. They are as follows

- To develop an intelligent system using Artificial Intelligence for the automatic detection and classification of retinal diseases from medical images. This reduces dependency on manual diagnosis and improves efficiency.
- To implement Deep Learning techniques, particularly Convolutional Neural Networks (CNNs), for accurate feature extraction and disease classification from retinal images.
- To perform effective image preprocessing techniques such as resizing, normalization, and noise reduction to enhance image quality and improve model performance.
- To integrate heatmap visualization methods (such as Grad-CAM) to provide interpretability by highlighting important regions in the retinal image used for prediction.
- To achieve high accuracy and reliability in detecting eye diseases like cataract, glaucoma, and diabetic retinopathy, ensuring better diagnostic support.

- To design a scalable and efficient system that can be deployed in real-world environments, especially in rural or resource-limited areas for early disease detection.

3.3 Functional Requirements

The system must provide intelligent eye disease detection and healthcare support by analyzing retinal images and offering diagnosis using Artificial Intelligence techniques. It should allow users to easily upload retinal images, receive disease predictions, view visual explanations, and access additional healthcare services through a simple and user-friendly interface.

- Eye Disease Detection and classification
- Heatmap Visualization
- Hospital Locator
- Appointment Booking
- General Health Routine
- User Interface

3.3.1 Actors

- User
The primary actor who interacts with the system by uploading retinal images and viewing the predicted results. The user can also access general recommendations, view nearby hospitals, and book appointments.
- Admin
The admin manages the overall system, including monitoring uploaded data, maintaining the database, and ensuring smooth functioning of the application. The admin may also update system features and manage hospital or recommendation data.
- Doctor/HealthcareProvider
The doctor reviews the results generated by the system and provides expert consultation if required. They may also handle appointments booked through the system and validate the diagnosis.

3.3.2 Use cases

- **Upload Retinal Image**
The user uploads a retinal image through the web interface for analysis. This is the initial step required to start the disease detection process.
- **Preprocess Image**
The system automatically processes the uploaded image by performing resizing, normalisation, and noise removal to improve image quality.
- **Disease Detection and Classification**
The system uses Deep Learning models to analyse the image and classify it into different eye diseases.
- **Generate Heatmap Visualization**
The system creates a heatmap to highlight important regions in the retinal image that influence the prediction.
- **Display Result**
The predicted disease, along with confidence score and heatmap, is displayed to the user in an understandable format.
- **View General Routine**
Based on the detected disease, the system provides suggestions such as precautions and eye care routines.
- **View Nearby Hospitals**
The system displays nearby hospitals or eye care centres for further medical consultation.
- **Book Appointment**
The user can schedule an appointment with a doctor through the system for further diagnosis and treatment.

3.4 Non-Functional Requirements:

- **Performance** - The system should provide fast processing and quick prediction results with minimal delay to ensure efficient usage.
- **Accuracy and Reliability** - The system should maintain high accuracy in detecting and classifying retinal diseases and produce consistent results.
- **Usability** - The system should have a simple and user-friendly interface so that users can easily upload images and view outputs.

- **Security** - The system should ensure proper protection of user data and medical images to maintain privacy and confidentiality.
- **Scalability** - The system should handle increasing data and user load without affecting performance or response time.
- **User Friendliness** – The interface should be simple, clear, and easy to use so that farmers can easily access all system functionalities without technical difficulty.

3.4.1 Hardware Specifications

- **Computer / Server** – Runs the ML model and backend computations.
- **Database Storage** – Stores user responses, test results, and progress data.
- **User Devices** – Web-based interface accessible on desktops, tablets and mobile devices.

Computer / Server

- **Processor**
Intel Core i5 / i7 or Apple M1/M2 (or higher) to efficiently handle model execution and backend computations.
- **RAM**
Minimum 8 GB RAM (16 GB recommended) for smooth processing of large image data and model operations.
- **Storage**
At least 256 GB SSD to store datasets, trained models, and system files with faster access speed.
- **GPU**
Optional but recommended GPU (NVIDIA for Windows or Apple Silicon GPU for Mac) to accelerate Deep Learning model training and inference.



Fig. 3.4.1.1 Intel Processor Chip

Database Storage

- **Database Technology**

The system uses SQLite as a lightweight database for storing retinal images, prediction results, and user-related data. It is easy to integrate and suitable for small to medium-scale applications.

- **Backend Integration**

PHP is used for server-side scripting to handle database operations such as storing data, retrieving results, and managing user interactions between the frontend and the database.

- **Storage Capacity**

A minimum of 256 GB storage is recommended to store datasets, model outputs, and user data efficiently.

- **Data Management**

The system ensures efficient data handling, including insertion, retrieval, and updating of records for smooth application performance.



Fig. 3.4.1.2 SQLite Logo

User Devices

- **Device Types**

The system can be accessed using desktops, laptops (Windows/macOS), tablets, providing flexibility for users to interact with the application from different platforms.

- **Display and Input**

Devices must support a clear display to view retinal images and results, along with input methods such as keyboard, mouse, or touch for easy interaction.

- **Internet Connectivity**

A stable internet connection is required to upload retinal images, process data, and receive prediction results from the server.

- **Browser Compatibility**

The system should be accessible through standard web browsers such as Chrome, Safari, or Edge, ensuring compatibility across different devices.



Fig. 3.4.1.3. laptop

3.4.2 Software Specifications

Operating System	:	Windows or macOS
Coding language	:	Python
Framework	:	TensorFlow, Keras, and OpenCV
Frontend	:	HTML, CSS and JavaScript
Algorithm	:	CNN

- OpenStreetMap API – Used to locate nearby hospitals and display them on the map.

Windows or macOS

The system is compatible with both Windows and macOS platforms, ensuring flexibility in development and deployment environments. It supports smooth execution of Python-based machine learning applications along with web-based interfaces. Users can easily run the system without major configuration issues on either operating system. The compatibility allows developers to work across different devices and setups

efficiently. It also ensures that the application can be accessed and tested in diverse environments. This cross-platform support improves usability and adaptability of the system.

For Windows, versions such as Windows 10 or above are recommended to ensure stable performance and compatibility with required tools. For macOS, systems with Intel processors or Apple Silicon chips like M1 or M2 are suitable for running the application. Both platforms support essential technologies such as Python, PHP, and modern web browsers. They also allow installation of required libraries and frameworks used in Deep Learning. This ensures that the system performs consistently across platforms. Overall, the use of Windows and macOS enhances the portability and efficiency of the application.



Fig. 3.4.2.1 Windows / mac OS logo

Python

Python is a high-level, interpreted programming language widely used in the fields of data science, machine learning, and web development. It is known for its simple and readable syntax, which makes it easy for developers to write and understand code efficiently. Python supports multiple programming paradigms, including object-oriented and functional programming. Its flexibility and ease of use make it a preferred choice for developing intelligent systems. In this project, Python serves as the primary programming language for implementing the entire application.

Python provides a rich ecosystem of libraries and frameworks that support Machine Learning and image processing tasks. Libraries such as TensorFlow, Keras, and OpenCV enable efficient model development, training, and image preprocessing. Python also supports data handling libraries like NumPy and Pandas, which are useful for managing datasets. Its integration capabilities allow smooth communication

between the frontend, backend, and database components. This makes Python highly suitable for building end-to-end machine learning applications.

Another advantage of Python is its strong community support and extensive documentation, which helps developers solve problems quickly. It is compatible with multiple platforms such as Windows and macOS, ensuring flexibility in development and deployment. Python also supports rapid prototyping, allowing developers to test and improve models efficiently. Its scalability makes it suitable for both small and large-scale applications. Overall, Python plays a crucial role in the successful implementation of this project.



Fig. 3.4.2.2 Python Language logo

TensorFlow, Keras, and OpenCV

TensorFlow is an open-source library widely used for building and training machine learning and Deep Learning models. It provides a flexible platform for designing neural networks and handling large-scale computations efficiently. TensorFlow supports both CPU and GPU execution, which helps in faster model training and inference. It offers various tools and APIs that simplify model development and deployment. In this project, TensorFlow is used as the core framework for implementing the deep learning model.

Keras is a high-level deep learning API that runs on top of TensorFlow and simplifies model development. It provides an easy-to-use interface for building neural networks with minimal code. Keras supports rapid prototyping, making it suitable for experimenting with different model architectures. It includes built-in functions for layers, activation functions, and optimizers. In this project, Keras is used to design

and train the Convolutional Neural Network (CNN) model efficiently.

OpenCV (Open Source Computer Vision Library) is used for image processing and computer vision tasks. It provides various functions for handling and manipulating images, such as resizing, filtering, and noise reduction. OpenCV helps in preprocessing retinal images before feeding them into the model. It improves image quality and enhances important features required for accurate prediction. In this project, OpenCV plays a key role in preparing input data for the deep learning model.



Fig. 3.4.2.3 TensorFlow and Keras logo

Convolutional Neural Network

A Convolutional Neural Network (CNN) is a deep learning algorithm commonly used for image recognition and computer vision tasks. It automatically extracts important features from images without manual feature selection. In this project, CNN is used to analyze crop leaf images and detect plant diseases.

CNN consists of layers such as convolution layers, pooling layers, and fully connected layers. The convolution layer detects patterns like edges and textures, the pooling layer reduces the image size while keeping important features, and the fully connected layer classifies the image into different disease categories.

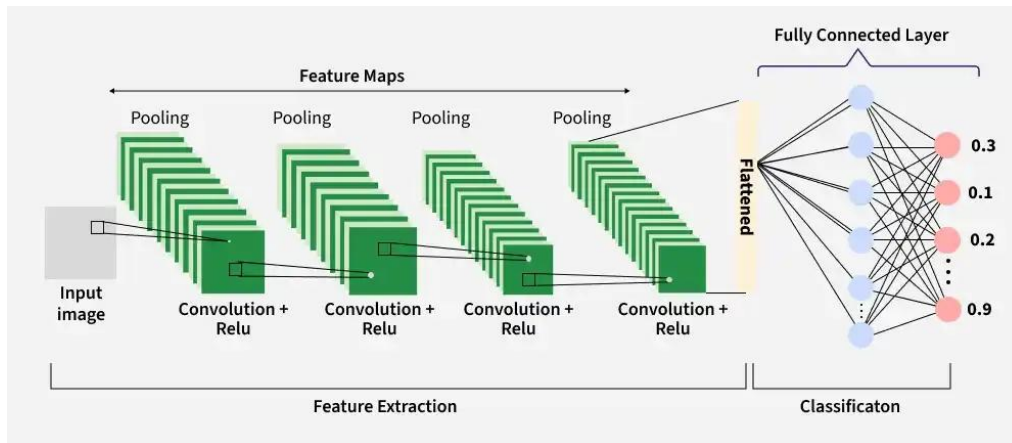


Fig. 3.4.2.4 CNN

HTML, CSS and JavaScript

HTML (HyperText Markup Language) is the standard language used to create the structure of web pages. It defines the layout of elements such as headings, forms, buttons, and image upload sections in the application. HTML is responsible for organizing the content displayed to the user in a clear and structured manner. In this project, it is used to design the interface for uploading retinal images and displaying results. It acts as the backbone of the web-based user interface.

CSS (Cascading Style Sheets) is used to style and enhance the appearance of the web pages. It controls the layout, colors, fonts, and overall design of the user interface. CSS helps in making the application visually appealing and user-friendly. It ensures that the interface is responsive and works well on different devices such as desktops, tablets, and mobiles. In this project, CSS is used to improve the presentation of the output and overall user experience. JavaScript is a scripting language used to add interactivity and dynamic behavior to web pages. It enables features such as form validation, image preview, and real-time updates without reloading the page. JavaScript helps in handling user actions and improving the responsiveness of the application. It also allows communication between the frontend and backend components. In this project, JavaScript is used to enhance user interaction and make the system more efficient and interactive.



Fig. 3.4.2.5 Layered approach of HTML, CSS and JS

OpenStreetMap API

OpenStreetMap is an open-source mapping platform that provides geographic data and location-based services. In this project, the OpenStreetMap API is used to detect the user's location and display nearby agro stores on the map, helping farmers easily find agricultural shops for seeds, fertilizers, and pesticides.



Fig. 3.4.2.6 Open Street Map Logo

3.5 ANALYSIS

The analysis phase defines the requirements of the system, independent of how these requirements will be accomplished. This phase defines the problem that the customer is trying to solve. The deliverable result at the end of this phase is a requirement document. Analysis object model is represented by class and object diagrams. Analysis focuses on producing a model of the system, called the Analysis model, which is correct, complete, consistent, and verifiable. The analysis model is

composed of three individual models: The Functional Model represented by use cases and scenarios, the Analysis Object Model, represented by class and object diagrams, and the Dynamic Model, represented by state chart and sequence diagrams. The analysis model represents the system under development from the user's point of view. The analysis object model is a part of the analysis and focuses on the individual concepts that are manipulated by the system, their properties and their relationships.

3.5.1 Entity Objects

The Analysis object model consists of entity, boundary and control objects. Entity objects represent the persistent information tracked by the system. Participating objects form the basis of the analysis model.

3.5.2 Boundary Objects

Boundary object is the object used for interaction between the user and the system. Moreover, it is an interface used to communicate with the system. Boundary objects represent the system interface with the actors. In each use case, each actor interacts with at least one boundary object. The boundary object collects the information from the actor and translates into an interface model from that can be used by the objects and by the control objects.

3.5.3 Control Objects

Control objects are responsible for coordinating entity objects and boundary objects. A control object is created at the beginning of the use cases and ceases to exist at its end. Control objects usually do not have a concrete counterpart in the real world. Control object is responsible for collecting information from the boundary objects and dispatching it to entity object. Here only authorized users can operate, and details obtained are verified. The control objects in this project are admission, result, placement, company and eligibility details.

3.5.4 Object Interaction

Interaction diagrams model the behaviour of use cases by describing the way groups of objects interact to complete the task. The two kinds of interaction diagrams are sequence and collaboration diagrams. Sequence diagrams generally show the sequence of events that occur.

3.5.5 Object Behaviour

State chart diagrams are used to describe the behaviour of a system. They define different states of an object during its lifetime. Each diagram usually represents objects of a single class and tracks the different states of its objects through the system and these states are changed by events. State chart diagram describes the flow of control from one state to another state. State chart diagrams are very important for describing the states. States can be identified as the condition of objects when an event occurs.

SYSTEM DESIGN

4. SYSTEM DESIGN

System Design is the process of defining the architecture, components, modules, interfaces, and data of a system to satisfy the specified requirements. In System design, developers:

- Define design goals of the project
- Decompose the system into smaller sub systems
- Design hardware/software strategies
- Design persistent data management strategies
- Design global control flow strategies
- Design access control policies and
- Design strategies for handling boundary conditions.

System design is not algorithmic. It is composed of several activities. They are:

- Identify Design Goals
- Design the initial subsystem decomposition
- Refine the subsystem decomposition to address the design goals.

System Design is the transformation of the analysis model into a system design model. Developers define the design goals of the project and divide the system into smaller modules such as Crop Disease Detection, Crop Recommendation, Agro Store Locator, and Voice Advisory modules. These modules work together to provide a complete agricultural assistance system for farmers.

Developers also select strategies for building the system, such as the software and hardware platform on which the system will run, the database management strategy using Firebase, the global control flow of the application, and the access control for user interaction. The result of the system design is a model that includes a clear description of these strategies, subsystem decomposition, and UML diagrams that represent the structure and functioning of the Smart Agriculture Assistance System.

4.1 Design Goals

Design goals are the qualities that the system should focus on during development.

- **UserFriendly:** The system is easy to use and understand, allowing farmers to easily upload crop images and receive recommendations.

- **Reliability:** The system provides accurate disease detection and reliable farming suggestions.
- **Efficiency:** The system processes crop images quickly and provides results in a short time.

4.2 System Architecture

As the complexity of systems increases, the specification of the system decomposition is critical. Moreover, subsystem decomposition is constantly revised whenever new issues are addressed. Subsystems are merged into alone subsystem, a complex subsystem is split into parts, and some subsystems are added to take care of new functionality. The first iterations over the subsystem decomposition can introduce drastic changes in the system design model.

4.3 System Design

The system is designed as a web-based application that integrates image processing and Deep Learning techniques for retinal disease detection. It consists of a user interface for uploading retinal images, a backend for processing, and a trained model for classification. The input image undergoes preprocessing before being passed to the model for prediction. The system also generates heatmap visualizations to highlight important regions in the image. Finally, the predicted result along with visualization is displayed to the user through the interface.

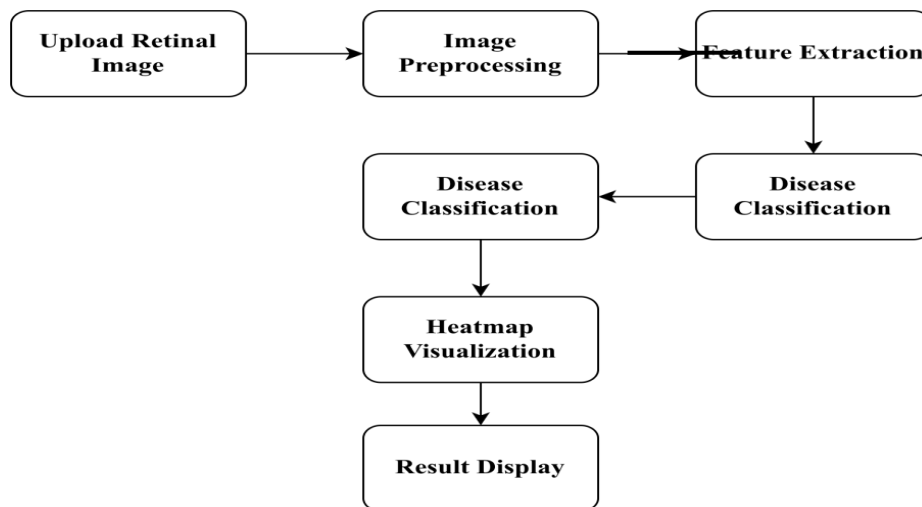


Fig. 4.3.1 System's Block Diagram

4.4 UML Diagrams

A UML diagram is a diagram based on the UML (Unified Modelling Language) with the purpose of visually representing a system along with its main actors, roles, actions, artifacts or classes, in order to better understand, alter, maintain, or document information about the system.

UML is a modern approach to modelling and documenting software. In fact, it's one of the most popular business process modelling techniques. It is based on diagrammatic representations of software components. As the old proverb says: "a picture is worth a thousand words". By using visual representations, we are able to better understand possible flaws or errors in software or business processes.

Mainly UML has been used as a general-purpose modelling language in the field of software engineering. However, it has now found its way into the documentation of several business processes or workflows. For example, activity diagrams, a type of UML diagram, can be used as a replacement for flowcharts. They provide both a more standardized way of modelling workflows as well as a wider range of features to improve readability.

4.4.1 Use Case Diagram

In the Unified Modelling Language(UML), a use case diagram can summarize the details of your system's users(also known as actors) and their interactions with the system. To build one, use a set of specialized symbols and connectors. An effective use case diagram can help the team discuss and represent:

- Scenarios in which system or application interacts with people, organizations, or external systems.
- Goals that your system or application helps those entities (known as actors) achieve.
- The scope of the system.

UML use case diagrams are ideal for:

- Representing the goals of system-user interactions.

- Defining and organizing functional requirements in a system.
- Specifying the context and requirements of a system.
- Modelling the basic flow of events in a use case.

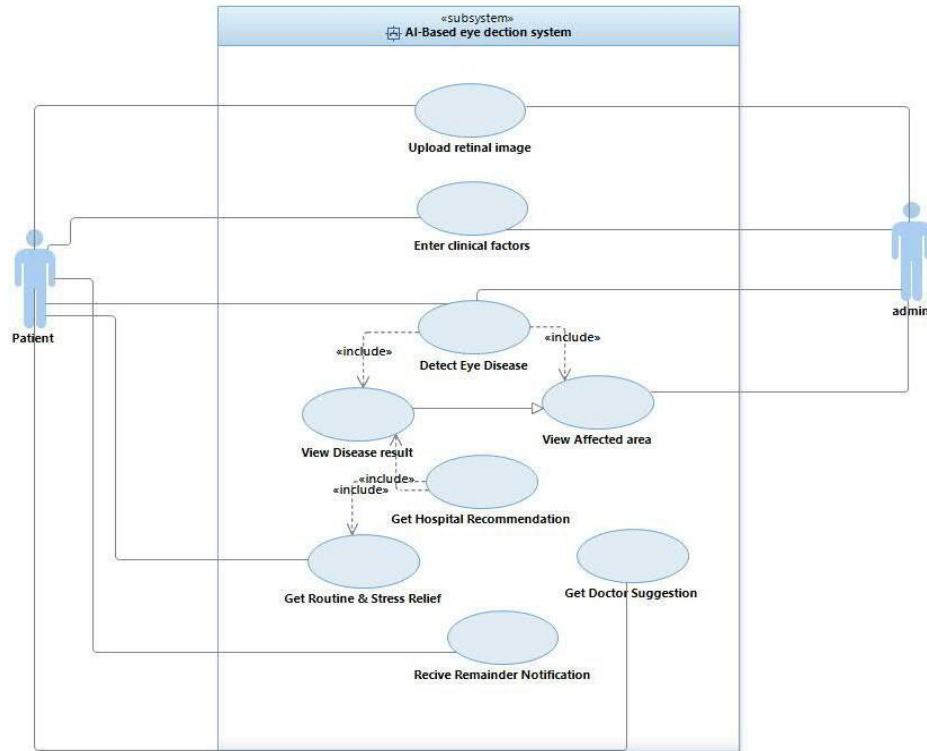


Fig. 4.4.1.1 Use Case Diagram

4.4.2 Class Diagram

Class diagrams are one of the most useful types of diagrams in UML as they clearly map out the structure of a particular system by modelling its classes, attributes, operations, and relationships between objects. It is a static diagram that represents the static view of an application. Class diagram is not only used for visualizing, describing, and documenting different aspects of a system but also for constructing executable code of the software application.

The class shape itself consists of a rectangle with three rows. The top row contains the name of the class, the middle row contains the attributes of the class, and the bottom section express the methods or operations that the class may use. Classes and subclasses are grouped together to show the static relationship between each object.

The purpose of the class diagram can be summarized as:

- Analysis and design of the static view of an application.
- Describe responsibilities of a system.
- Base for component and deployment diagrams.
- Forward and reverse engineering.

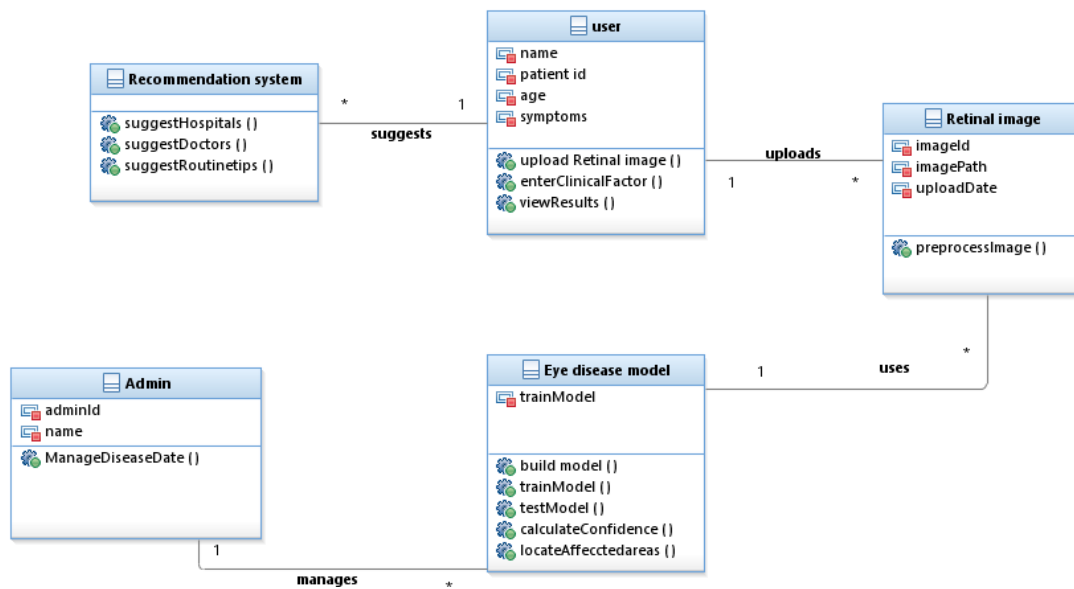


Fig. 4.4.2.1 Class Diagram

4.4.3 Activity Diagram

Activity diagram is basically a flowchart to represent the flow from one activity to another activity. The activity can be described as an operation of the system.

The control flow is drawn from one operation to another. This flow can be sequential, branched, or concurrent. Activity diagrams deal with all type of flow control by using different elements such as fork, join, etc. Activity is a particular operation of the system. Activity diagrams are not only used for visualizing the dynamic nature of a system, but they are also used to construct the executable system by using forward and reverse engineering techniques.

The purpose of an activity diagram can be described as

- Draw the activity flow of a system.
- Describe the sequence from one activity to another.

- Describe the parallel, branched and concurrent flow of the system.

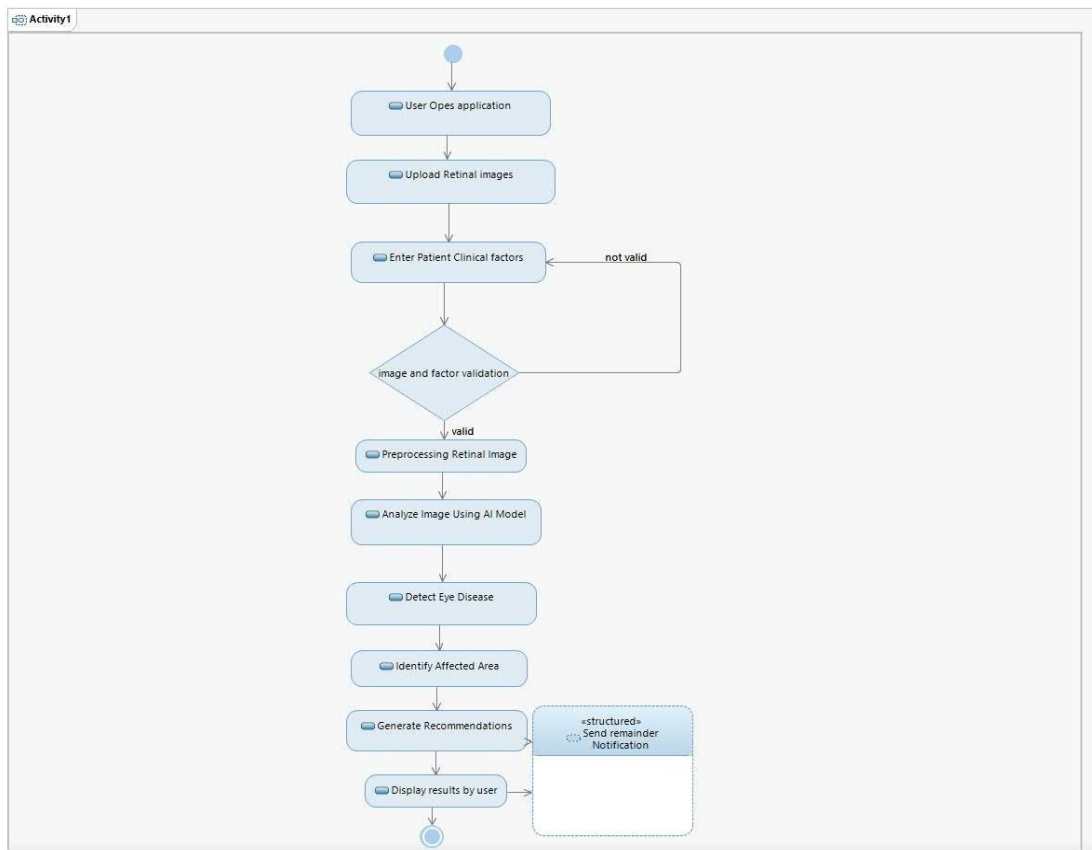


Fig. 4.4.3.1 Activity Diagram

4.4.4 Sequence Diagram

A sequence diagram is the most commonly used interaction diagram. It simply depicts interaction between objects in a sequential order i.e. the order in which these interactions take place. We can also use the terms event diagrams or event scenarios to refer to a sequence diagram. Sequence diagrams describe how and in what order the objects in a system function. These diagrams are widely used by businessmen and software developers to document and understand requirements for new and existing systems.

Sequence diagrams can be useful references for businesses and other organizations. Purpose of sequence diagrams are:

- Represent the details of a UML use case.
- Model the logic of a sophisticated procedure, function, or operation.
- See how objects and components interact with each other to complete a process.

- Plan and understand the detailed functionality of an existing or future scenario.

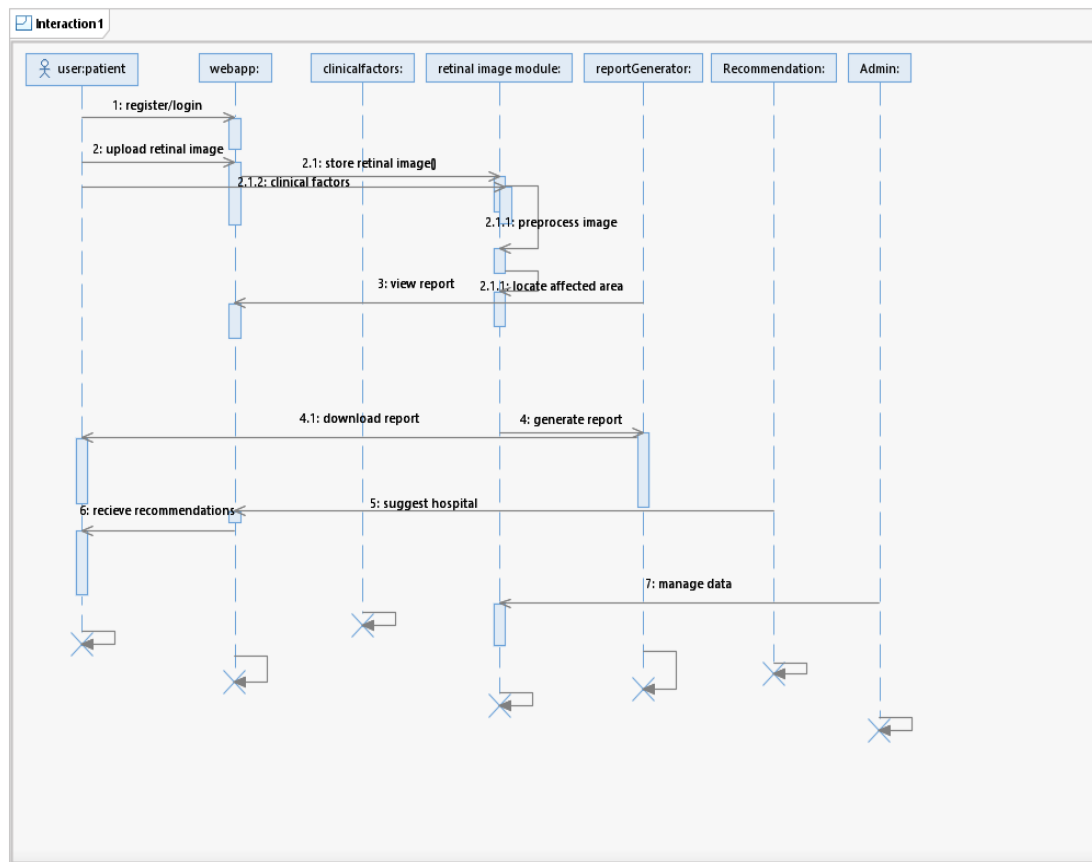


Fig. 4.4.4.1 Sequence Diagram

4.5 Data Flow Diagrams

A Data Flow Diagram (DFD) maps out the flow of information for any process or system. It uses defined symbols like rectangles, circles and arrows, plus short text labels, to show data inputs, outputs, storage points and the routes between each destination. Data flowcharts can range from simple, even hand-drawn process overviews, to in-depth, multi-level DFDs that dig progressively deeper into how the data is handled.

4.5.1 DFD Level-0

DFD Level 0 is also called as Context Diagram. It's a basic overview of the whole system or process being analysed or modelled. It's designed to be an at-a-glance view, showing the system as a single high-level process, with its relationship to external entities. In our project the external entity is driver.

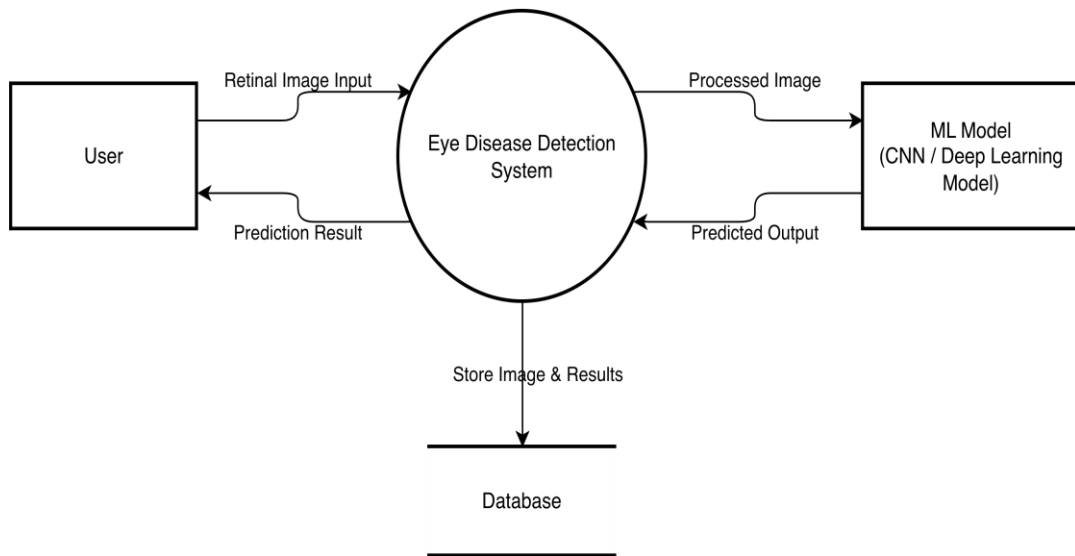


Fig. 4.5.1.1 DFD Level 0

4.5.2 DFD Level-1

DFD Level 1 provides a more detailed breakout of pieces of the Context Level Diagram. You will highlight the main functions carried out by the system, as you break down the high-level process of the Context Diagram into the subprocesses.

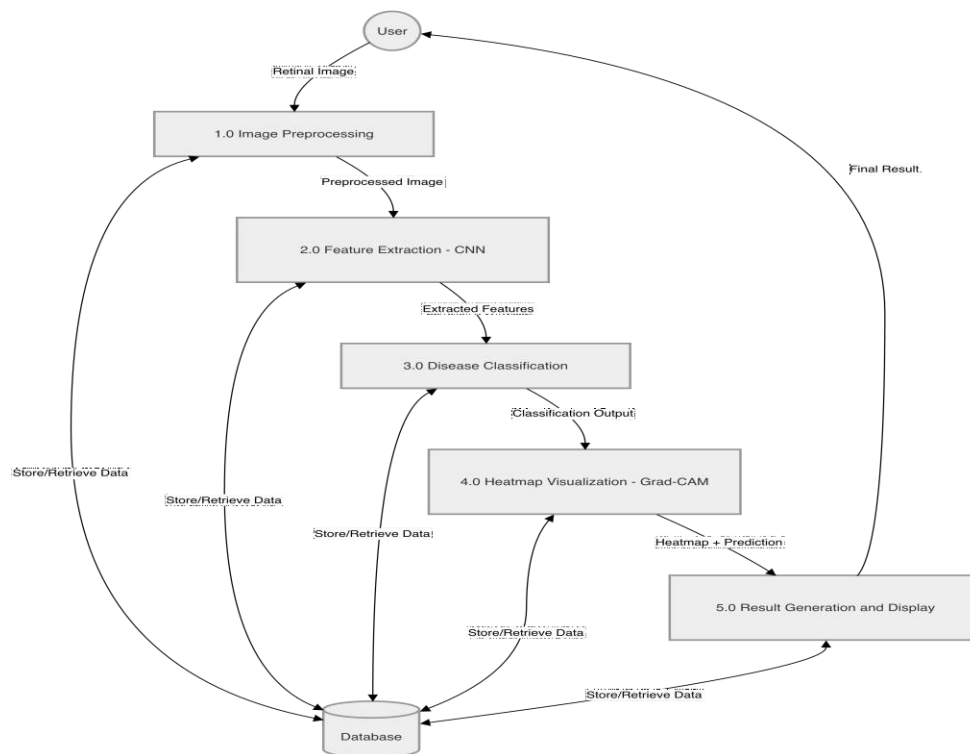


Fig 4.5.2.1 DFD Level 1

4.6 Flow Chart

A flowchart is a type of diagram that represents an algorithm, workflow or process. A flowchart can also be defined as a diagrammatic representation of an algorithm. The flowchart shows the steps as boxes of various kinds, and their order by connecting the boxes with arrows. This diagrammatic representation illustrates a solution model to a given problem. Flowcharts are used in analysing, designing, documenting or managing a process or program in various fields.

The flowchart of the system begins with the user uploading a retinal image through the web interface. The uploaded image is then passed to the preprocessing stage, where it undergoes operations such as resizing, normalization, and noise reduction to improve its quality. After preprocessing, the image is forwarded to the Deep Learning model, where important features are extracted automatically.

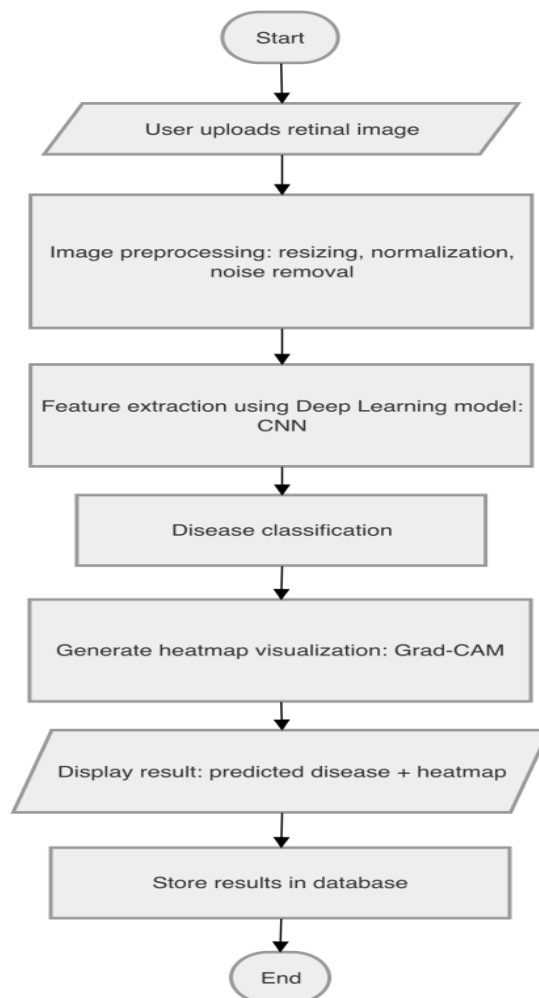


Fig. 4.6.1 Flow chart of the system

Next, the extracted features are used for disease classification, where the system predicts the type of eye disease such as cataract, glaucoma, or diabetic retinopathy. Once the prediction is made, the system generates a heatmap visualization using techniques like Grad-CAM to highlight the regions of the image that influenced the decision.

Finally, the system displays the predicted result along with the heatmap to the user through the interface. The results and images are also stored in the database for future reference. This step-by-step process ensures accurate, efficient, and interpretable detection of retinal diseases.

IMPLEMENTATION

5. IMPLEMENTATION

The objective of the coding or programming phase is to translate the design of the system produced during the design phase into code in a given programming language, which can be executed by a computer and that performs the computation specified by the design.

The coding phase affects both testing and maintenance. The goal of coding is not to reduce the implementation cost, but the goal should be to reduce the cost of later phases.

5.1 Coding Approach

There are two major approaches for coding any software system. They are Top- Down approach and bottom-up approach.

Bottom-up Approach can best suit for developing the object-oriented systems. During system design phase, we decompose the system into an appropriate number of subsystems, for which objects can be modelled independently. These objects exhibit the way the subsystems perform their operations.

Once objects have been modelled, they are implemented by means of coding. Even though related to the same system as the objects are implemented of each other, the Bottom-Up approach is more suitable for coding these objects. In this approach, we first do the coding of objects independently and then we integrate these modules into one system to which they belong.

This code will first allow the admin to login with a valid user id and password. After this, admin will select the admission or academic module and enter all the students' details branch wise. TPO will select placement module and enter the details based on academic performance. Reports are generated accordingly and can be viewed by teaching staff and students.

5.2 Information Handling

Any software system requires some amount of information during its operation selection of appropriate data structures can help us to produce the code so that objects of the system can better operate with the available information decreased complexity.

5.3 Programming Style

Programming style deals with act of rules that a programmer must follow so that the characteristics of coding such as Traceability, Understandability, Modifiability, and Extensibility can be satisfied. In this current system, we followed the coding rules for naming the variables and methods.

The code is structured in such a way that each module, such as image processing, recommendation, and user interface, is easy to understand and modify. This improves the overall quality of the application and makes future enhancements easier.

5.4 Verification and Validation

Verification is the process of checking the product built is right. Validation is the process of checking whether the right product is built. During the Development of the system, Coding for the object has been thoroughly verified from different aspects regarding their design, in the way they are integrated etc. The various techniques that have been followed for validation discussed in testing the current system.

Validations applied to the entire system at two levels:

Form level Validation:

All user inputs such as crop image uploads and data entries are validated before processing. The system ensures that required inputs are provided and displays appropriate error messages if any input is missing or incorrect.

Field level Validation:

Each individual input field is validated to prevent errors such as invalid data or incomplete entries. The system provides clear alerts and messages to guide the user in correcting mistakes.

These validation techniques ensure that the system works accurately, reduces errors, and provides a smooth user experience.

5.5 Implementation of Project

The implementation of the proposed system is carried out as a web-based application that integrates frontend technologies, backend processing, and Deep Learning models for efficient retinal disease detection. The system is developed using HTML, CSS, and JavaScript for the user interface, while Python is used for implementing the machine learning model. The backend server is managed using PHP, and XAMPP is used as the local server environment to run Apache and manage database operations. SQLite/MySQL is used to store user data, retinal images, and prediction results.

The system begins with a user-friendly interface where users can upload retinal images for analysis. Once an image is uploaded, it is sent to the backend where preprocessing is performed using OpenCV. This includes resizing, normalization, and noise reduction to enhance the image quality. The processed image is then passed to a trained Convolutional Neural Network (CNN) model built using TensorFlow and Keras, which extracts features and classifies the image into different eye diseases such as cataract, glaucoma, and diabetic retinopathy.

After classification, the system generates a prediction along with a confidence score. To improve transparency, heatmap visualization using Grad-CAM is applied to highlight the important regions of the retinal image that influenced the model's decision. The final output, including the predicted disease and heatmap, is displayed to the user through the web interface. All results and uploaded images are stored in the database for future reference and analysis.

In addition to disease detection, the system provides extended functionalities to enhance user support. Based on the detected disease, the system offers general health recommendations such as precautions, lifestyle changes, and basic care suggestions. It also includes a feature to display nearby hospitals or eye care centers where users can seek professional consultation. Furthermore, the system provides an appointment booking option, allowing users to schedule visits with doctors directly through the interface.

The integration of XAMPP ensures smooth local deployment and testing of the application by managing server-side operations and database connectivity. PHP handles communication between the frontend and backend, while the database efficiently stores and retrieves user and prediction data. Overall, the implementation combines machine

learning, web technologies, and healthcare support features to create a complete and practical solution for early eye disease detection and assistance.

5.6 System Workflow & Process Flow

- The user opens the mobile application and selects the required service.
- The system accepts user input such as crop image or crop details.
- The input is processed and analysed by the system to generate results.
- The system provides appropriate outputs such as disease information, recommendations.
- The user can access voice guidance and view results easily through the application.

TESTING

6. TESTING

Testing is the process of finding differences between the expected behaviour specified by system models and the observed behaviour of the system. Testing is a critical role in quality assurance and ensuring the reliability of development and these errors will be reflected in the code, so the application should be thoroughly tested and validated.

Unit testing finds the differences between the object design model and its corresponding components. Structural testing finds differences between the system design model and a subset of integrated subsystems. Functional testing finds differences between the use case model and the system.

Finally, performance testing, finds differences between non-functional requirements and actual system performance. From modelling point of view, testing is the attempt of falsification of the system with respect to the system models. The goal of testing is to design tests that exercise defects in the system and to reveal problems.

6.1 Testing Activities

Testing a large system is a complex activity and like any complex activity. It has to be broke into smaller activities. Thus, incremental testing was performed on the project i.e., components and subsystems of the system were tested separately before integrating them to form the subsystem for system testing.

6.2 Testing Types Unit Testing

Unit testing focuses on the building blocks of the software system that is the objects and subsystems. There are three motivations behind focusing on components. First unit testing reduces the complexity of overall test activities allowing focus on smaller units of the system, second unit testing makes it easier to pinpoint and correct faults given that few components are involved in the rest. Third unit testing allows parallelism in the testing activities, that is each component are involved in the test. Third unit testing allows parallelism in the testing activities that is each component can be tested independently of one another. The following are some unit testing techniques.

6.2.1 Equivalence testing:

It is a black box testing technique that minimizes the number of test cases. The possible inputs are partitioned into equivalence classes and a test case is selected for each class.

6.2.2 Boundary testing:

It is a special case of equivalence testing and focuses on the conditions at the boundary of the equivalence classes. Boundary testing requires that the elements be selected from the edges of the equivalence classes.

6.2.3 Path testing:

It is a white box testing technique that identifies faults in the implementation of the component the assumption here is that exercising all possible paths through the code at least once. Most faults will trigger failure. This acquires knowledge of source code.

Integrating Testing

Integration testing defects faults that have not been detected. During unit testing by focusing on small groups on components two or more components are integrated and tested and once tests do not reveal any new faults, additional components are added to the group. This procedure allows testing of increasing more complex parts on the system while keeping the location of potential faults relatively small. I have used the following approach to implements and integrated testing.

Top-down testing strategy unit tests the components of the top layer and then integrated the components of the next layer down. When all components of the new layer have been tested together, the next layer is selected. This was repeated until all layers are combined and involved in the test.

Validation Testing

The systems completely assembled as package, the interfacing have been uncovered and corrected, and a final series of software tests are validation testing. The validation testing is nothing but validation success when system functions in a manner that can be reasonably expected by the customer. The system validation had done by series of Black-box test methods.

System Testing

1. System testing ensures that the complete system compiles with the functional requirements and non-functional requirements of the system, the following are some system testing activities.
2. Functional testing finds differences between the functional between the functional requirements and the system. This is a black box testing technique. Test cases are divided from the use case model.
3. Pilot testing the system is installed and used by a selected set of users – users exercise the system as if it had been permanently installed.
4. Functional testing was performed to check whether the system correctly detects diseases, provides recommendations, and displays nearby stores.
5. Acceptance testing, I have followed benchmarks testing in a benchmark testing the client prepares a set of test cases represent typical conditions under which the system operates. In our project, there are no existing benchmarks.

Installation testing, the system is installed in the target environment.

Black Box Testing also known as Behavioural Testing, is a software testing method in which the internal structure/design/implementation of the item being tested is not known to the tester. These tests can be functional or non-functional, though usually functional. This method is named so because the software program, in the eyes of the tester, is like a black box; inside which one cannot see. This method attempts to find errors in the following categories:

- Incorrect or missing functions
- Interface errors
- Errors in data structures or external database access
- Behaviour or performance errors
- Initialization and termination errors

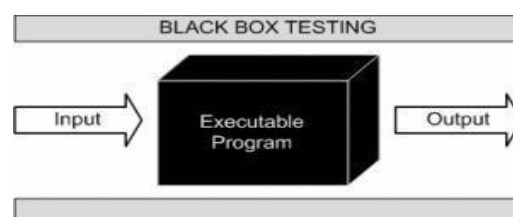


Fig. 6.2.1 Black Box Testing

6.3 Test cases

Test Case #	Test Case Description	Test Data	Expected Result	Actual Result	Pass/Fail
1	Upload valid retinal image	Retinal Image (JPEG/PNG)	Image uploaded successfully	Image uploaded successfully	PASS
2	Upload invalid file format	PDF/Text file	Error message displayed	Error message displayed	PASS
3	Image preprocessing	Retinal Image	Image resized and normalized	Image processed correctly	PASS
4	Disease detection model	Preprocessed Image	Correct disease predicted	Disease predicted correctly	PASS
5	Heatmap generation	Retinal Image	Heatmap highlighting affected area	Heatmap generated	PASS
6	Display result to user	Prediction Output	Disease + confidence shown	Result displayed correctly	PASS
7	Hospital recommendation feature	Disease detected	Nearby hospitals displayed	Hospitals shown	PASS

8	Appointment booking	User details	Appointment confirmed	Appointment booked	PASS
9	General Routine	User Details	Display General routine	Routine shown	PASS
10	Application display in pdf	Results + Heatmap	Generate pdf	Pdf shown	PASS

RESULTS

7. RESULTS

7.1 Home Page

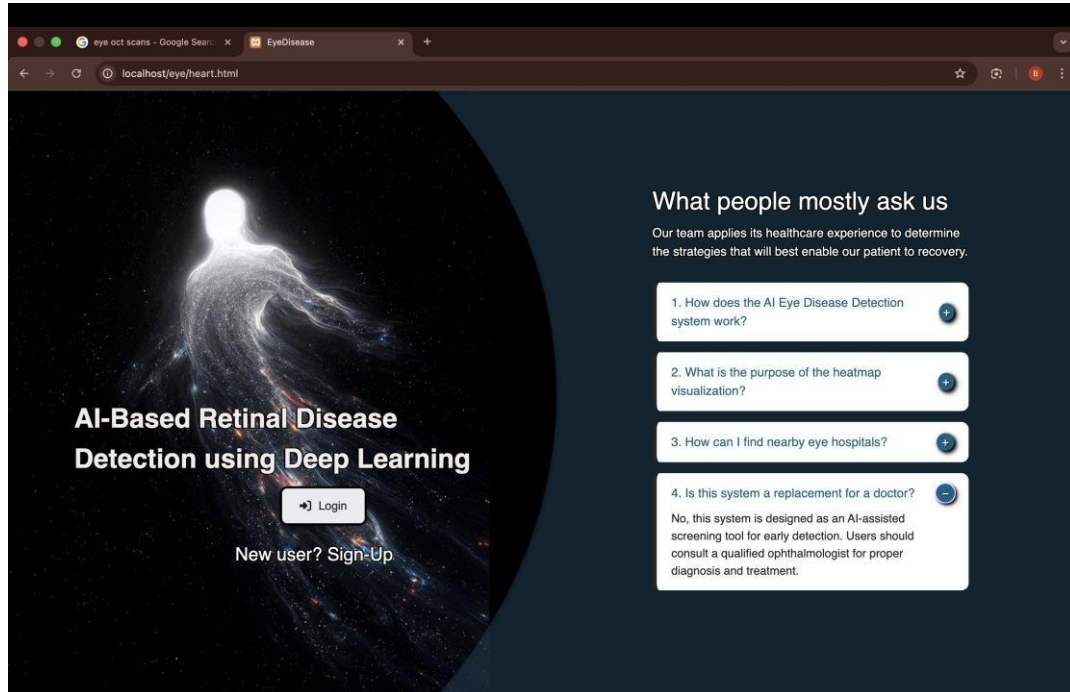


Fig. 7.1 Start Page

7.2 Login Page

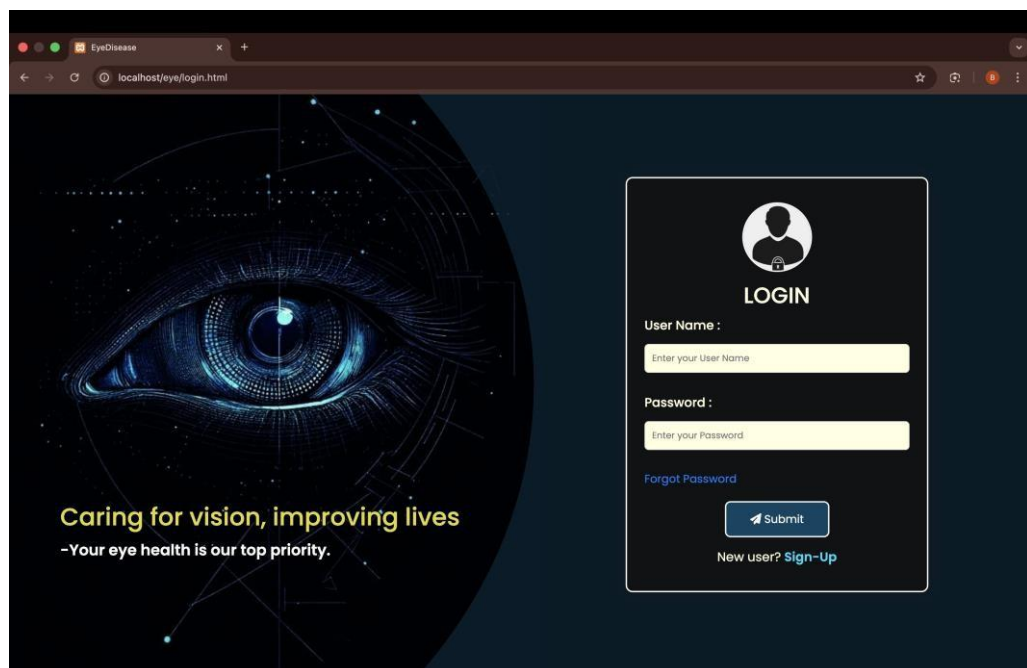


Fig. 7.2.1 Login Page

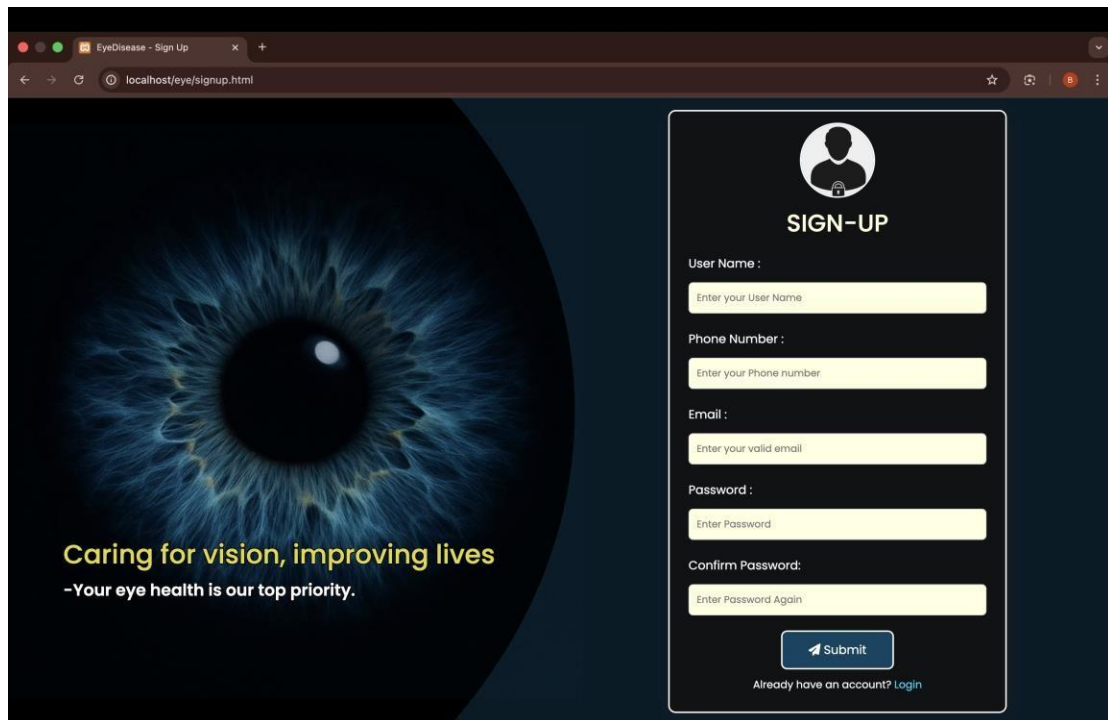


Fig. 7.2.2 Signup Page

7.3 Eye disease detection

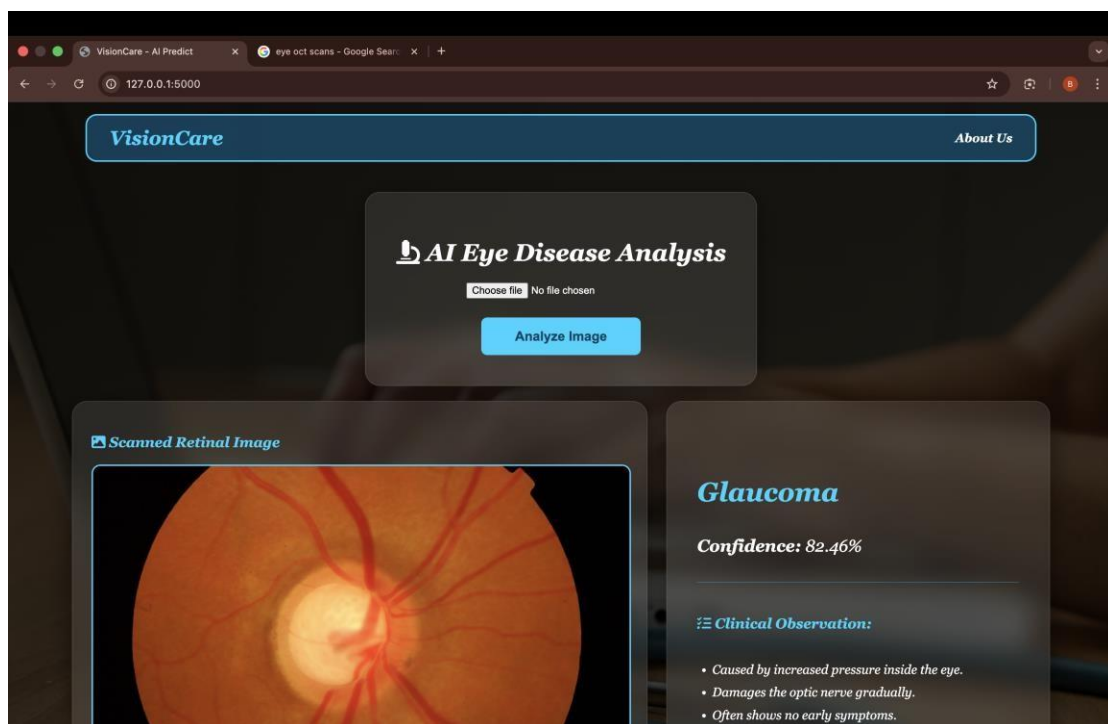


Fig. 7.3 Detection Page

7.4 Result

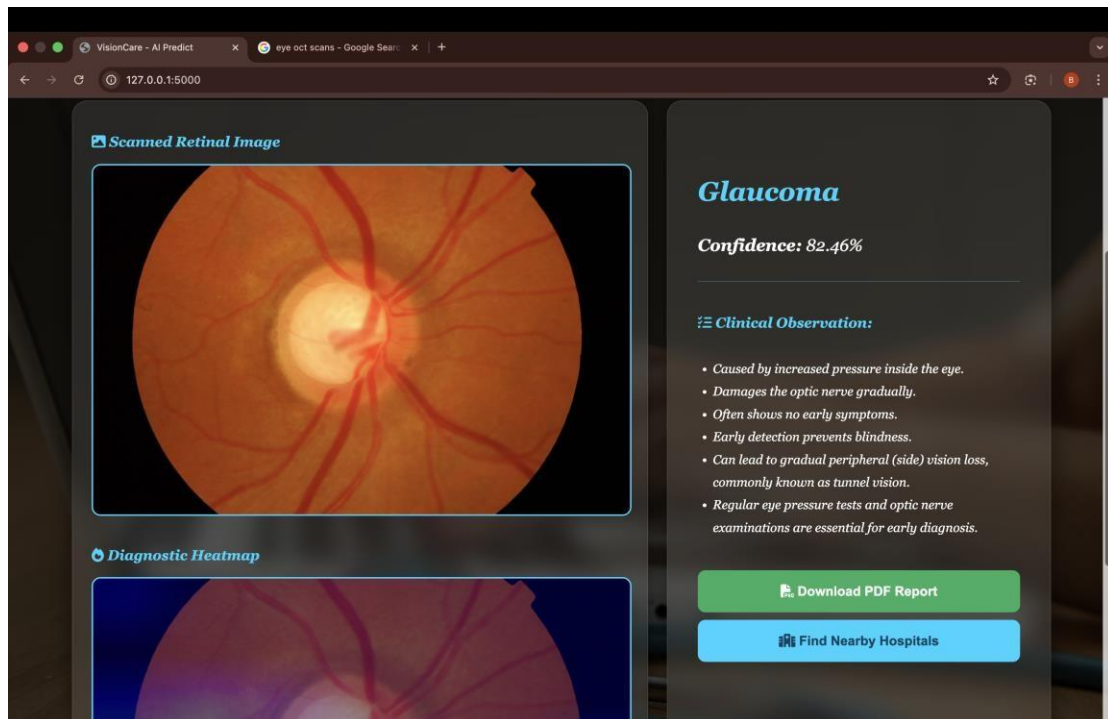


Fig. 7.4.1 Prediction

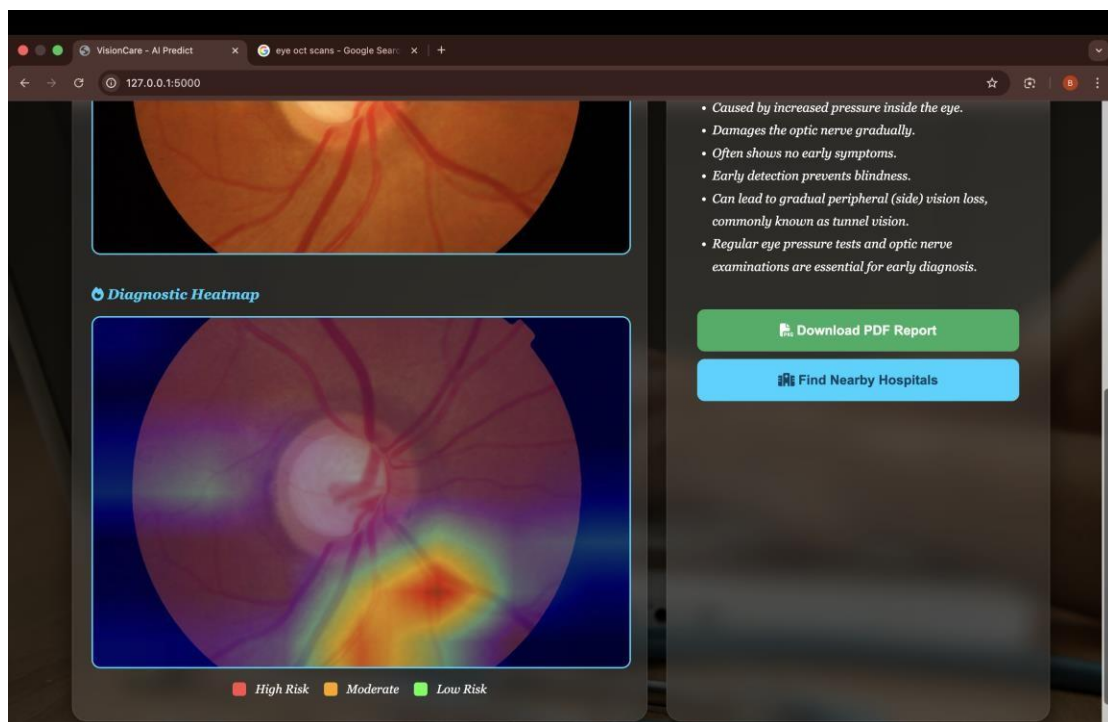


Fig. 7.4.2 Heatmap Result

7.5 Routine Page

The screenshot shows a web browser window with the URL `localhost/eye/routine.html`. The page has a dark blue header with the "VisionCare" logo and navigation links for "Home", "Hospitals", "Appointments", and a "Logout" button. The main content area is titled "Daily Eye Health Routine" with a heart icon. It contains several input fields and dropdown menus for user information: "Your Age:" (text input with placeholder "Enter Age"), "Dietary Preference:" (dropdown menu showing "Vegetarian"), "Daily Activity Level:" (dropdown menu showing "Sedentary (Low)"), "Average Sleep (Hours):" (text input with placeholder "Hours/Night"), "Current Stress Level:" (dropdown menu showing "Low"), "Pre-existing Sight Issues?" (dropdown menu showing "No"), and "Primary Health Condition:" (dropdown menu showing "Normal"). At the bottom of the form is a large blue button labeled "Generate Personalized Plan".

Fig. 7.5.1 Routine Page

The screenshot shows the result page after clicking the "Generate Personalized Plan" button. The page has a dark blue background with a light blue dashed border around the content area. At the top of this area is a blue button labeled "Generate Personalized Plan". Below it, the text "Your Suggested Routine:" is followed by a list of two items: "30 mins brisk walking" and "Stay hydrated". The next section is "Saturday's Diet Plan (Vegetarian, Normal):" followed by a list of three items: "Paratha + curd", "Paneer curry", and "Roti + sabzi". The following section is "Tips for Better Heart Health:" followed by a list of three items: "Stay hydrated with 8+ glasses of water", "Eat plenty of fiber", and "Avoid skipping meals". The final section is "Recommended Workouts:" followed by a list of three items: "Blink slowly 10 to 15 times to reduce dryness.", "rule: Every 20 minutes, look 20 feet away for 20 seconds", and "Eye rolling: Gently roll eyes clockwise and anti-clockwise."

Fig. 7.5.2 Routine Result

7.6 Nearest Hospitals

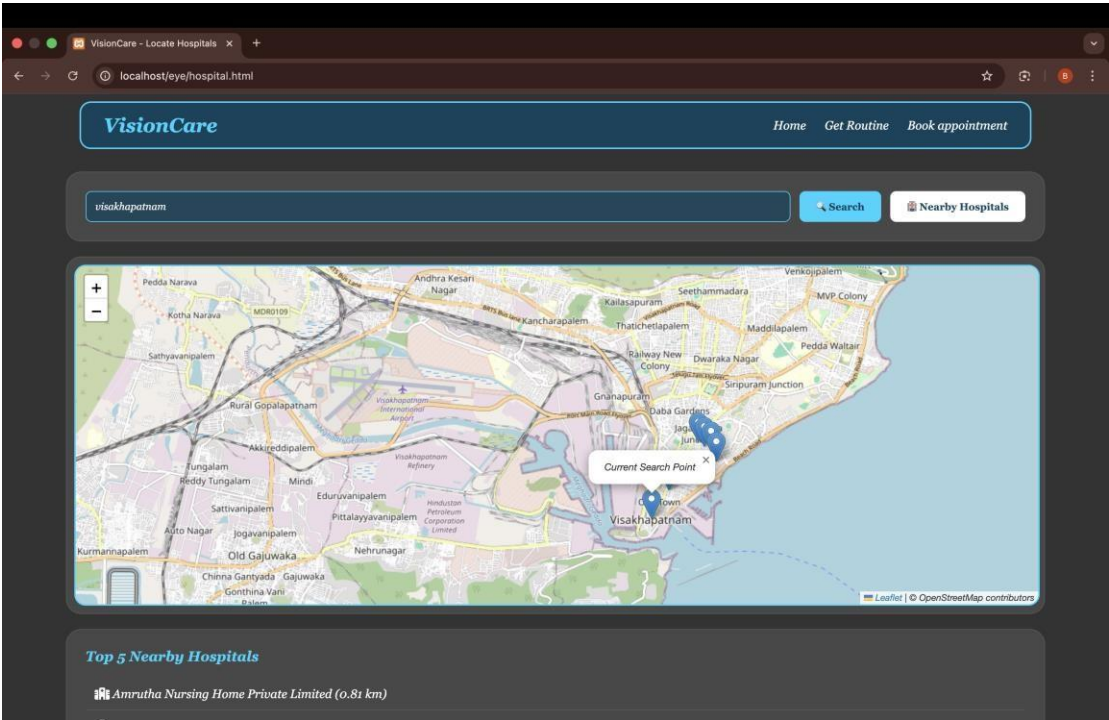


Fig. 7.6.1 Nearest Hospital

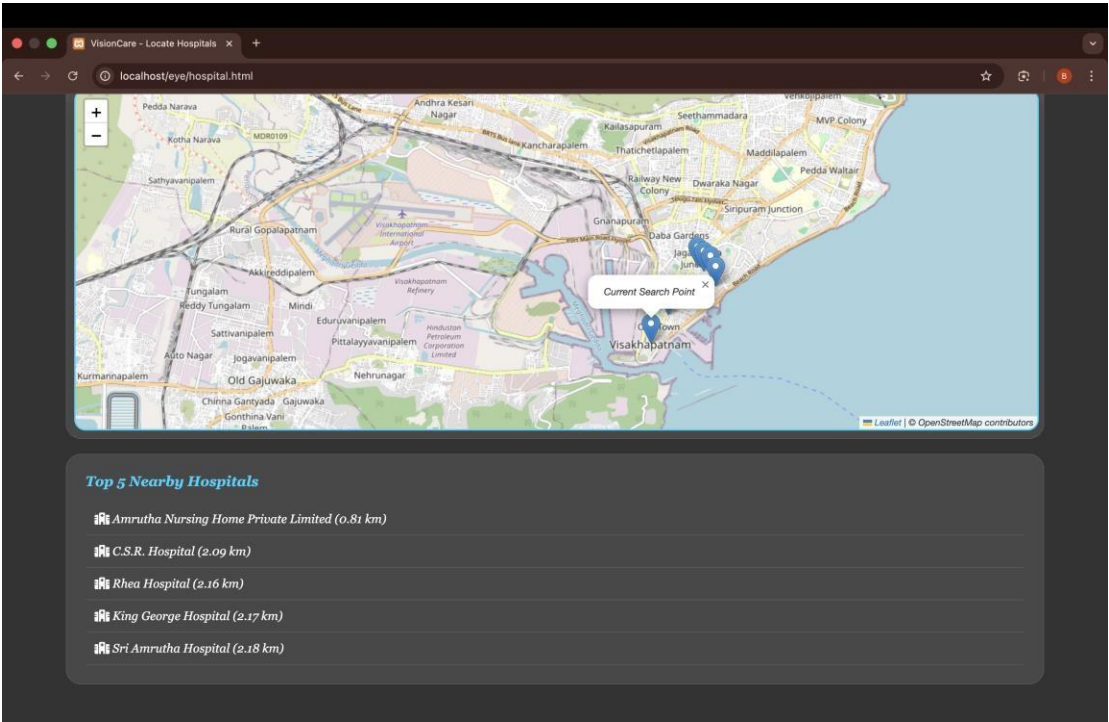
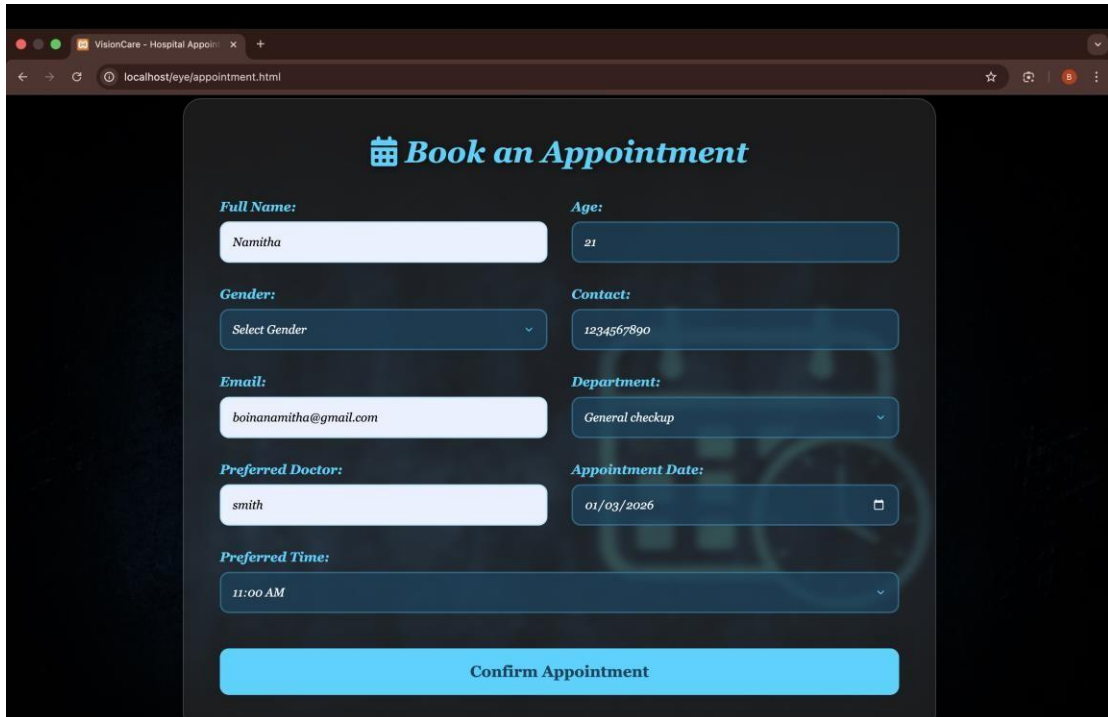


Fig. 7.6.2 Nearest Hospital Result

7.7 Book Appointment



The screenshot shows a web browser window with the URL `localhost/eye/appointment.html`. The page has a dark background with a light blue header area containing the title "Book an Appointment" with a calendar icon. Below the title, there are several form fields arranged in two columns. The left column contains fields for "Full Name:" (text input with "Namitha"), "Gender:" (dropdown menu with "Select Gender"), "Email:" (text input with "boinanamitha@gmail.com"), "Preferred Doctor:" (text input with "smith"), and "Preferred Time:" (dropdown menu with "11:00 AM"). The right column contains fields for "Age:" (text input with "21"), "Contact:" (text input with "1234567890"), "Department:" (dropdown menu with "General checkup"), and "Appointment Date:" (text input with "01/03/2026" and a calendar icon). At the bottom of the form is a large blue button labeled "Confirm Appointment".

Fig. 7.7.1 Appointment Page

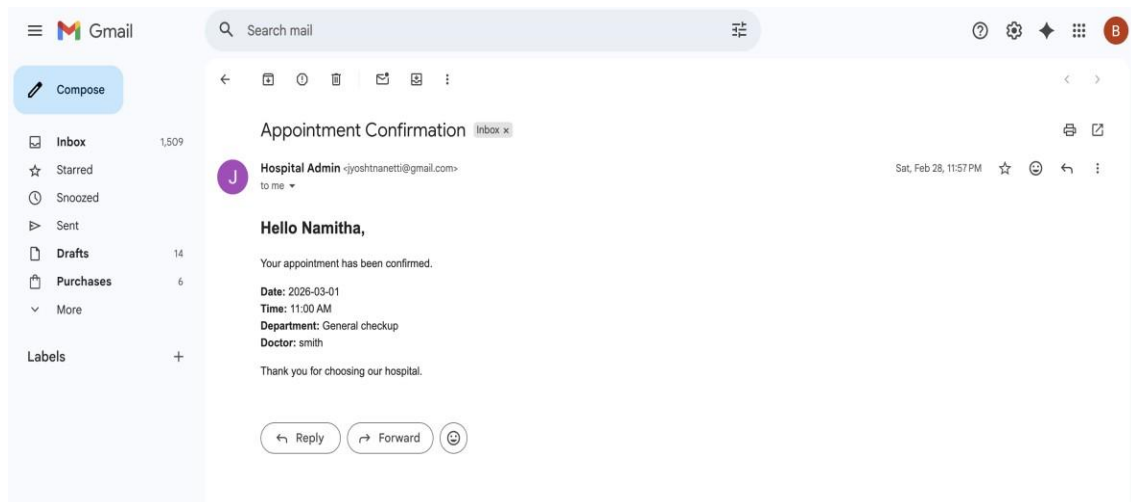


Fig. 7.7.2 Appointment Result Mail

SAMPLE CODE

8.SAMPLE CODE

8.1 Code for Model Training

```
import tensorflow as tf

from tensorflow.keras.applications import ResNet50

from tensorflow.keras.models import Model

from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout

from tensorflow.keras.preprocessing.image import ImageDataGenerator

from tensorflow.keras.optimizers import Adam

import matplotlib.pyplot as plt


DATASET_PATH = "dataset"

IMG_SIZE = (224, 224)

BATCH_SIZE = 32

EPOCHS = 25

NUM_CLASSES = 4


datagen = ImageDataGenerator(
    rescale=1.0/255.0,
    validation_split=0.2)


train_data = datagen.flow_from_directory(
    DATASET_PATH,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='training')


val_data = datagen.flow_from_directory(
    DATASET_PATH,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
```



```

subset='validation')
base_model = ResNet50(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3))

for layer in base_model.layers[:-30]:
    layer.trainable = False

for layer in base_model.layers[-30:]:
    layer.trainable = True

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
output = Dense(NUM_CLASSES, activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=output)

model.compile(
    optimizer=Adam(learning_rate=0.00001), # smaller LR for fine-tuning
    loss='categorical_crossentropy',
    metrics=['accuracy'])
model.summary()

history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=EPOCHS)
model.save("eye_disease_model.h5")
print("Model training complete and saved!")
plt.plot(history.history['accuracy'], label='Train Accuracy')

```

```
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

8.2 Code for detection

```
import os
import numpy as np
import tensorflow as tf
import cv2

from flask import Flask, render_template, request, send_file
from tensorflow.keras.preprocessing import image
from fpdf import FPDF

app = Flask(__name__)
UPLOAD_FOLDER = "static/uploads"
HEATMAP_FOLDER = "static/heatmaps"
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs(HEATMAP_FOLDER, exist_ok=True)

# Load Model
model = tf.keras.models.load_model("eye_disease_model.h5")
LAST_CONV_LAYER = "conv5_block3_out"
CLASS_NAMES = ["cataract", "diabetic_retinopathy", "glaucoma", "normal"]

disease_details = {
    "diabetic_retinopathy": [
        "Caused by damage to retinal blood vessels due to diabetes.",
        "May cause blurred vision or dark spots.",
        "Can lead to permanent vision loss if untreated.",
        "Control blood sugar and get regular eye checkups.",
        "In advanced stages, abnormal new blood vessels may grow on the retina.",
        "Early stages may not show noticeable symptoms, making regular screening essential."
    ],

```

```

    "glaucoma": [
        "Caused by increased pressure inside the eye.",
        "Damages the optic nerve gradually.",
        "Often shows no early symptoms.",
        "Early detection prevents blindness.",
        "Can lead to gradual peripheral (side) vision loss, commonly known as tunnel vision.",
        "Regular eye pressure tests and optic nerve examinations are essential for early diagnosis."
    ],
    "cataract": [
        "Clouding of the eye lens.",
        "Causes blurry or dim vision.",
        "Common in older adults.",
        "Treatable with a simple surgery.",
        "May cause increased sensitivity to light and glare, especially at night.",
        "Colors may appear faded or yellowish as the cataract progresses."
    ],
    "normal": [
        "No major eye disease detected.",
        "Vision appears healthy.",
        "Continue regular eye checkups.",
        "Maintain good eye care habits."
    ]
}

```

```

def preprocess_image(img_path):
    img = image.load_img(img_path, target_size=(224, 224))
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
    img_array = img_array / 255.0
    return img_array

def generate_gradcam(img_array, model, last_conv_layer_name):

```

```

grad_model = tf.keras.models.Model(
    [model.inputs],
    [model.get_layer(last_conv_layer_name).output, model.output]
)
with tf.GradientTape() as tape:
    conv_outputs, predictions = grad_model(img_array)
    class_index = tf.argmax(predictions[0])
    loss = predictions[:, class_index]

grads = tape.gradient(loss, conv_outputs)
pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
conv_outputs = conv_outputs[0]
heatmap = conv_outputs @ pooled_grads[..., tf.newaxis]
heatmap = tf.squeeze(heatmap)
heatmap = np.maximum(heatmap, 0) / (tf.math.reduce_max(heatmap) + 1e-10)
return heatmap.numpy()

def save_and_superimpose(img_path, heatmap, output_path):
    img = cv2.imread(img_path)
    heatmap = cv2.resize(heatmap, (img.shape[1], img.shape[0]))
    heatmap = np.uint8(255 * heatmap)
    heatmap = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)
    superimposed_img = cv2.addWeighted(img, 0.6, heatmap, 0.4, 0)
    cv2.imwrite(output_path, superimposed_img)

def is_retinal_image(img):
    """
    Stronger retinal validation:
    1. Red dominance
    2. Circular structure
    3. Dark borders
    """
    if img is None:

```

```

    return False

resized = cv2.resize(img, (224, 224))
gray = cv2.cvtColor(resized, cv2.COLOR_BGR2GRAY)

b, g, r = cv2.split(resized)
red_mean = np.mean(r)
green_mean = np.mean(g)
blue_mean = np.mean(b)

if red_mean < green_mean or red_mean < blue_mean:
    return False

circles = cv2.HoughCircles(
    gray,
    cv2.HOUGH_GRADIENT,
    dp=1.2,
    minDist=100,
    param1=50,
    param2=30,
    minRadius=60,
    maxRadius=120
)

if circles is None:
    return False

border = np.concatenate([
    gray[0:20, :].flatten(),
    gray[-20:, :].flatten(),
    gray[:, 0:20].flatten(),
    gray[:, -20:].flatten()
])

if np.mean(border) > 90:

```

```
return False
```

```
return True
```

```
@app.route("/", methods=["GET", "POST"])
```

```
def predict():
```

```
    prediction = None
```

```
    confidence = None
```

```
    image_path = None
```

```
    heatmap_path = None
```

```
    disease_info = None
```

```
    error_message = None
```

```
    if request.method == "POST":
```

```
        file = request.files.get("image")
```

```
        if file and file.filename != "":
```

```
            filepath = os.path.join(UPLOAD_FOLDER, file.filename)
```

```
            file.save(filepath)
```

```
            img = cv2.imread(filepath)
```

```
            if not is_retinal_image(img):
```

```
                if os.path.exists(filepath):
```

```
                    os.remove(filepath)
```

```
            return render_template(
```

```
                "index.html",
```

```
                error_message="Please upload a valid retinal image."
```

```
            )
```

```
            img_array = preprocess_image(filepath)
```

```
            preds = model.predict(img_array)
```

```
            class_index = np.argmax(preds[0])
```

```

confidence = round(float(np.max(preds[0]) * 100), 2)

prediction = CLASS_NAMES[class_index]
disease_info = disease_details.get(
    prediction.lower(),
    ["Consult a specialist."]
)

heatmap = generate_gradcam(img_array, model, LAST_CONV_LAYER)
heatmap_file_path = os.path.join(
    HEATMAP_FOLDER,
    "heatmap_" + file.filename
)

save_and_superimpose(filepath, heatmap, heatmap_file_path)
image_path = "/" + filepath.replace("\\", "/")
heatmap_path = "/" + heatmap_file_path.replace("\\", "/")

return render_template(
    "index.html",
    prediction=prediction,
    confidence=confidence,
    image_path=image_path,
    heatmap_path=heatmap_path,
    disease_info=disease_info,
    error_message=error_message )
img_array = preprocess_image(filepath)
preds = model.predict(img_array)

class_index = np.argmax(preds[0])
confidence = round(float(np.max(preds[0]) * 100), 2)

prediction = CLASS_NAMES[class_index]
disease_info = disease_details.get(prediction.lower(), ["Consult a specialist."])

```

```

heatmap = generate_gradcam(img_array, model, LAST_CONV_LAYER)
heatmap_file_path = os.path.join(HEATMAP_FOLDER, "heatmap_" + file.filename)
save_and_superimpose(filepath, heatmap, heatmap_file_path)

image_path = "/" + filepath.replace("\\", "/")
heatmap_path = "/" + heatmap_file_path.replace("\\", "/")

class_index = np.argmax(preds[0])
confidence = round(float(np.max(preds[0]) * 100), 2)
if os.path.exists(filepath):
    os.remove(filepath)

    return render_template(
        "index.html",
        error_message=error_message
    )
prediction = CLASS_NAMES[class_index]
disease_info = disease_details.get(prediction.lower(), ["Consult a specialist."])

heatmap = generate_gradcam(img_array, model, LAST_CONV_LAYER)
heatmap_file_path = os.path.join(HEATMAP_FOLDER, "heatmap_" +
file.filename)
save_and_superimpose(filepath, heatmap, heatmap_file_path)

image_path = "/" + filepath.replace("\\", "/")
heatmap_path = "/" + heatmap_file_path.replace("\\", "/")

return render_template(
    "index.html",
    prediction=prediction,
    confidence=confidence,
    image_path=image_path,
    heatmap_path=heatmap_path,

```



```

        disease_info=disease_info,
        error_message=error_message
    )

@app.route("/download_report", methods=["POST"])
def download_report():

    prediction = request.form.get("prediction", "Unknown")
    confidence = request.form.get("confidence", "0")
    image_path = request.form.get("image_path")
    heatmap_path = request.form.get("heatmap_path")
    disease_info = disease_details.get(
        prediction.lower(),
        ["Consult a specialist."]
    )

    if image_path:
        image_path = image_path.lstrip("/")
    if heatmap_path:
        heatmap_path = heatmap_path.lstrip("/")

    pdf = FPDF()
    pdf.set_auto_page_break(auto=True, margin=15)
    pdf.add_page()

    page_width = pdf.w - 20

    # Title
    pdf.set_font("Arial", 'B', 20)
    pdf.cell(page_width, 15, "Eye Disease Diagnostic Report", ln=True, align='C')
    pdf.ln(5)

    # Result Section

```

```

pdf.set_font("Arial", 'B', 14)
pdf.cell(page_width, 10, f"Disease: {prediction.upper()}", ln=True)
pdf.cell(page_width, 10, f"Confidence Score: {confidence}%", ln=True)
pdf.ln(5)

# Images
img_width = 80
img_height = 60
y_position = pdf.get_y()

try:
    if image_path and os.path.exists(image_path):
        pdf.image(image_path, x=10, y=y_position, w=img_width)

    if heatmap_path and os.path.exists(heatmap_path):
        pdf.image(heatmap_path, x=110, y=y_position, w=img_width)

except Exception as e:
    print("PDF Image Error:", e)

pdf.set_y(y_position + img_height + 15)

# Detailed Analysis
pdf.set_font("Arial", 'B', 14)
pdf.cell(page_width, 10, "Detailed Analysis:", ln=True)

pdf.set_font("Arial", "", 11)

for point in disease_info:
    pdf.multi_cell(page_width, 8, f"- {point}")
    pdf.ln(1)

pdf.ln(5)

```

```

pdf.set_font("Arial", 'B', 14)
pdf.cell(page_width, 10, "Heatmap Colour Legend:", ln=True)
pdf.set_font("Arial", "", 11)
legend_points = [
    "Red: High Risk Area",
    "Yellow: Moderate Risk Area",
    "Green: Low Risk Area"]

for point in legend_points:
    pdf.cell(page_width, 8, f"- {point}", ln=True)

pdf.ln(5)

report_path = "Diagnostic_Report.pdf"
pdf.output(report_path)

return send_file(report_path, as_attachment=True)
@app.route("/hospitals")
def hospitals():
    return render_template("hosp.html")

if __name__ == "__main__":
    app.run(debug=True)

```

Code for landing page

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="style.css">
    <link
                                                rel="stylesheet"
        href="https://cdn.jsdelivr.net/npm/bootstrap@4.4.1/dist/css/bootstrap.min.css"

```

```

integrity="sha384-
Vkoo8x4CGsO3+Hh xv8T/Q5PaXtkKtu6ug5TOeNV6gBiFeWPGFN9MuhOf23
Q9Ifjh" crossorigin="anonymous">
<link      rel="stylesheet"      href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css">
<style>
.rou {
    background-image: url("first.jpeg");
    background-size: 45% auto;
    background-position: left center;
    background-repeat: no-repeat;
    background-color: #000000;
    height: 100vh;
}
body {
    margin: 0;
    font-family: Georgia, serif;
    overflow: hidden;
}
.curve-section {
    position: relative;
    width: 100%;
    height: 100vh;
    display: flex;
}

.curve-shape {
    position: absolute;
    top: 0;
    right: 0;
    width: 100%;
    height: 100%;
    z-index: 1;

```

```

    filter: drop-shadow(2px 2px 2px rgb(35, 35, 37));
    /* THIS LINE MAKES BUTTONS WORK WITHOUT CHANGING DESIGN
    */
    pointer-events: none;
}

.curve-shape svg {
    width: 100%;
    height: 100%;
    display: block;
}

.faq-content {
    position: absolute;
    top: 50%;
    right: 25%;
    transform: translate(50%, -50%);
    padding: 40px;
    z-index: 2;
    width: 35%;
    max-width: 600px;
}

.faq-content h2 {
    font-size: 34px;
    margin-bottom: 10px;
    color: white;
    text-shadow: -1px -1px 0 #000, 1px -1px 0 #000, -1px 1px 0 #000, 1px 1px 0
    #000;
    font-family: 'Poppins', sans-serif;
}

```

```
.faq-content p {
    margin-bottom: 30px;
    color:white;
    text-shadow: -1px -1px 0 #000, 1px -1px 0 #000, -1px 1px 0 #000, 1px 1px 0
    #000;
    font-size: 17px;
}
```

```
.faq-item {
    background:white;
    border-radius: 10px;
    margin-bottom: 15px;
    padding: 15px 20px;
    border-left: 6px solid ;
    /* Re-enable clicks for FAQ buttons since curve-shape disabled them */
    pointer-events: auto;
}
```

```
.faq-question {
    display: flex;
    justify-content: space-between;
    align-items: center;
    font-size: 17px;
    cursor: pointer;
    color:rgb(3, 97, 128);
}
```

```
.faq-answer {
    margin-top: 10px;
    font-size: 16px;
    display: none;
}
```

```
.toggle-btn {
  background:rgb(3, 97, 128);
  border: none;
  color: white;
  font-size: 16px;
  width: 24px;
  height: 24px;
  border-radius: 50%;
  line-height: 24px;
  text-align: center;
  cursor: pointer;
  text-shadow: 1px 1px 1px black;
  box-shadow: 2px 2px 5px rgba(0, 0, 0, 0.7);
  filter: drop-shadow(2px 2px 2px black);
}
```

```
.highlight-wrapper {
  position: absolute;
  top: 50%;
  transform: translateY(-50%);
  width: 50%;
  z-index: 2;
  color:rgb(231, 219, 219);
  padding: 60px;
  border-radius: 12px;
  /* Re-enable clicks for Login/Signup links */
  pointer-events: auto;
}
```

```
.logo-title {
  font-size: 37px;
  font-family: 'Poppins', sans-serif; /*  correct */
  font-weight: 600; /* better than normal */
}
```

```

display: flex;
align-items: center;
margin-bottom: 10px;
margin-top: 250px;
color: rgb(242, 237, 237);
text-shadow: -1px -1px 0 #000,
             1px -1px 0 #000,
             -2px 1px 0 #000,
             1px 1px 0 #000;
margin-left: 5%;
}

```

```

.login-btn {
background-color: rgb(235, 236, 239);
color: rgb(31, 32, 33);
padding: 12px 24px;
border: 3px solid black;
border-radius: 10px;
font-size: 16px;
display: inline-flex;
align-items: center;
gap: 10px;
cursor: pointer;
text-decoration: none;
transition: background 0.3s ease;
box-shadow: 2px 2px 5px rgba(0, 0, 0, 0.5);
margin-left: 51%;
}

```

```

.login-btn:hover {
background-color: rgb(5, 68, 97);
color: white;
}

```



```
.signup-link1 {
    margin-left:41%;
    margin-top:4%;
    font-size: 25px;
    color:white;
    text-shadow: -1px -1px 0 #000, 1px -1px 0 #000, -1px 1px 0 #000, 1px 1px 0
    #000;
}
```

```
.signup-link1 a :hover {
    background-color: rgb(5, 68, 97);
    color:white;
}
```

```
</style>
<title>EyeDisease</title>
</head>
<body>
<div class="rou">
<div class="curve-section">
<div class="curve-shape">
<svg viewBox="0 0 100 100" preserveAspectRatio="none">
<path d="M100,0 L40,0 C55,30 55,70 40,100 L100,100 Z" fill="rgba(5, 68, 97,
0.4)" />
</svg>
</div>

<div class="faq-content">
<h2>What people mostly ask us</h2>
<p>Our team applies its healthcare experience to determine the strategies that will
```

best enable our patient to recovery.</p>

<div class="faq-item">

<div class="faq-question">1. How does the AI Eye Disease Detection system work? <button class="toggle-btn">+</button></div>

<div class="faq-answer">

The system uses a ResNet50-based Convolutional Neural Network to analyze uploaded retinal images. It classifies them into Cataract, Diabetic Retinopathy, Glaucoma, or Normal based on learned visual patterns.

</div>

</div>

<div class="faq-item">

<div class="faq-question">2. What is the purpose of the heatmap visualization? <button class="toggle-btn">+</button></div>

<div class="faq-answer">

The heatmap is generated using Grad-CAM to highlight important regions in the retinal image. It shows which areas influenced the model's prediction, improving transparency and trust.

</div>

</div>

<div class="faq-item">

<div class="faq-question">3. How can I find nearby eye hospitals? <button class="toggle-btn">+</button></div>

<div class="faq-answer">

The system includes a built-in hospital locator that displays nearby clinics on a map. Users can search by location and get directions to specialized medical centers.

</div>

</div>

<div class="faq-item">

<div class="faq-question">4. Is this system a replacement for a doctor? <button class="toggle-btn">+</button></div>

```
<div class="faq-answer">
```

No, this system is designed as an AI-assisted screening tool for early detection. Users should consult a qualified ophthalmologist for proper diagnosis and treatment.

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="highlight-wrapper">
```

```
<div class="logo-title">AI-Based Retinal Disease Detection using Deep Learning</div>
```

```
<a href="login.html" class="login-btn">
```

```
<i class="fas fa-sign-in-alt"></i> Login
```

```
</a>
```

```
<h1 class="signup-link1">New user? <a href="signup.html" style="color:white;">Sign-Up</a></h1>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<script>
```

```
const toggleButtons = document.querySelectorAll('.toggle-btn');
```

```
toggleButtons.forEach(button => {
```

```
button.addEventListener('click', () => {
```

```
const answer = button.parentElement.nextElementSibling;
```

```
const isVisible = answer.style.display === 'block';
```

```
document.querySelectorAll('.faq-answer').forEach(ans => ans.style.display = 'none');
```

```
document.querySelectorAll('.toggle-btn').forEach(btn => btn.textContent = '+');
```

```
if (!isVisible) {
```

```
answer.style.display = 'block';
```

```
button.textContent = '−';
```

```
}
```

```
    });  
  });  
</script>  
</body>  
</html>
```

CONCLUSION

9. CONCLUSION

The proposed system successfully demonstrates the application of Artificial Intelligence and Deep Learning techniques for the detection and classification of retinal diseases. By using Convolutional Neural Networks, the system is able to accurately analyze retinal images and identify conditions such as cataract, glaucoma, and diabetic retinopathy. The integration of preprocessing techniques further enhances the quality of input data, improving the overall performance of the model.

In addition to disease detection, the system provides heatmap visualization to improve interpretability and help users understand the model's predictions. The inclusion of additional features such as general health recommendations, hospital locator, and appointment booking makes the system more practical and user-friendly. These features extend the application beyond simple prediction to a complete healthcare support system.

Overall, the project offers an efficient, accurate, and accessible solution for early eye disease detection. It has the potential to assist medical professionals and benefit individuals, especially in rural and resource-limited areas. This system highlights the importance of AI-driven solutions in improving healthcare services and reducing the risk of vision loss through early diagnosis.

REFERENCES

10. REFERENCES

- [1] S. Authors *et al.*, “AI-Driven Eye Disease Classification Model Using Denoising Autoencoders and Whale Optimization,” *Applied Sciences*, vol. 13, no. 20, 2023. [Online]. Available: <https://www.mdpi.com/2076-3417/13/20/11437>
- [2] “A Review of Eye Disease Detection Using Deep Learning Techniques,” 2024. [Online]. Available: <https://share.google/jVeK8sGd5uW4vTsZV>
- [3] A. Krizhevsky, I. Sutskever, and G. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” 2012.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE CVPR*, 2016.
- [5] “Eye Diseases Classification Dataset,” Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/gunavenkatdoddi/eye-diseases-classification>
- [6] R. R. Selvaraju *et al.*, “Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization,” in *Proc. ICCV*, 2017.
- [7] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *Proc. ICLR*, 2015.
- [8] Y. LeCun, Y. Bengio, and G. Hinton, “Deep Learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [9] J. Gulshan *et al.*, “Development and Validation of a Deep Learning Algorithm for Detection of Diabetic Retinopathy in Retinal Fundus Photographs,” *JAMA*, 2016.
- [10] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” 2014.

